Accountability in Certificate Transparency and Variants

Abstract

Certificate Transparency (CT) aims to reduce the trust required in Certificate Authorities (CAs) within the TLS certificate ecosystem. It is supported by all major browsers. The protocol obliges all CAs to record the certificates they issue in a public log, which itself is monitored for compliance and consistency by third parties. Given this complex set of checks between the four roles—CA, loggers, monitor but also the end user's client—it is very hard to provide a precise account of how CT eliminates trust assumptions in exchange for complex infrastructure. Analyses both in the Dolev-Yao paradigm and the computational paradigm only regard a very simplified model and feature definitions adapted specifically to CAs, essentially capturing design features rather than the target property.

The present paper posits accountability as the main goal of CT and presents a thorough analysis in the Dolev-Yao model. We start with the vanilla PKI and, step by step, move to CT, finally analyzing proposed extensions for SCT Auditing and Gossiping. We show that plain CT relies on an honest log, but provides accountability under this assumption. Furthermore, we show that the SCT Auditing extension can eliminate this assumption, while the Gossiping extension cannot.

1 Introduction

The web relies heavily on its Public Key Infrastructure (PKI) to ensure secure communication and authenticitcy. The web PKI utilizes certificates issued by Certificate Authorities (CAs) to establish trusted connections between clients and websites, as specified in the TLS protocol [1]. Historically, CAs act as trusted third parties that assert (most importantly) the hostname-key mapping presented in the certificate. CAs are expected to follow rigorous guidelines, the CA/Browser Forum Baseline Requirements [2] to prevent wrongly issued certificates. There have been multiple instances where CAs violated these requirements, which at worst allowed attackers to perform man-in-the-middle attacks [3], [4], [5], [6], [7]. Closely monitoring the actions of CAs has been a long-standing challenge with hundreds of CAs and tens of millions of daily issued certificates [8]. To address this issue, the Certificate Transparency (CT) protocol was introduced [9], [10], which enjoys ever-growing adoption in more and more browsers and most recently found its way into the Android ecosystem [11]. CT requires all issued certificates to be published in append-only logs, that allow anyone to monitor and audit the certificates issued by CAs. This effectively distributes the trust assumption from a single CA to the CA and involved logs. CT does not prevent malicious behavior, but it aims at making it visible early on. The steps after detection of such an attack are outside the scope of CT, but CT logs should provide the necessary information to identify the attack and attribute it to the responsible CA. A successful example of this was the case against Symantec, where multiple rogue certificates were found in CT logs [6], [7]. We posit accountability, the ability to identify and correctly attribute misbehavior to the responsible parties, as the key property of CT.

We apply automated analysis to CT using Tamarin in the Dolev-Yao model. Tamarin has built-in support to prove accountability: we define a set of *case tests* that aim to identify parties violating the security property. This mechanism is part of the protocol, although often underspecified. It is implemented through the parties in the protocol, without an outside view of every action. We verify, formally and automatically, that this identification contains all parties it should and no party it should not. Our model of CT is the most detailed to date, and the first formal model able to describe how to keep logs or monitors accountable. Our results are the first to consider corruption scenarios involving all three roles: CA, log, and monitor. Compared to results outside formal methods, we are not only the first to be able to hold logs or monitors accountable, but the first to guarantee that (a) all responsible parties are held accountable (not just one) and (b) no innocent party is blamed. Our goal is to show that if authenticity is violated, CT provides the necessary information to identify the misbehaving parties causing the violation. First, we find attacks refuting this claim. Second, we formulate assumptions that prove it but are unrealistic. Third, we show how additional infrastructure (in CT, or, later, in extension of CT) may ensure these assumptions. We perform this detailed analysis for basic CT, for SCT auditing (as implemented in the Chrome browser) and STH gossiping (an academic proposal).

Organisation. In Section 2, we start with necessary background, including the CT standard and its proposed extensions. Section 3 gives a brief overview of modeling using Tamarin and further details on the accountability notion we use in this work. We then start incrementally introducing our model and consider the PKI setting first in Section 4, where we show that authenticity can be violated and accountable authenticity achieved only given an external validator, which is not applicable in practice. Section 5 then adds Certificate Transparency to the PKI model. We show that authenticity is still violated and that accountable authenticity requires transparency, which can be violated too. In Sections 6 and 7 we consider two additional proposals, SCT auditing and gossiping, and attempt to prove the same properties. We find that SCT auditing requires the fewest additional assumptions to achieve accountability. Section 8 recapitulates our findings and presents details on the proofs for every considered property and model. Section 9 summarizes other works that consider CT and accountability.

2 Certificate Transparency Protocol

Root CAs form the root of trust in the plain PKI setting, which are implicitly trusted by all parties. They sign certificates for intermediate CAs that are then transitively trusted as well and sign certificates for end entities. In order to verify a certificate of an end entity, the entire chain of certificates up to a root CA certificate is verified (chain-of-trust). There are hundreds of root CAs, each being the single point of failure. The requirements for a root CA are very strict. Following the baseline requirements, they undergo yearly legal audits and publish all their signed intermediate CAs to the public CCADB database [2], [18]. In order to distribute trust

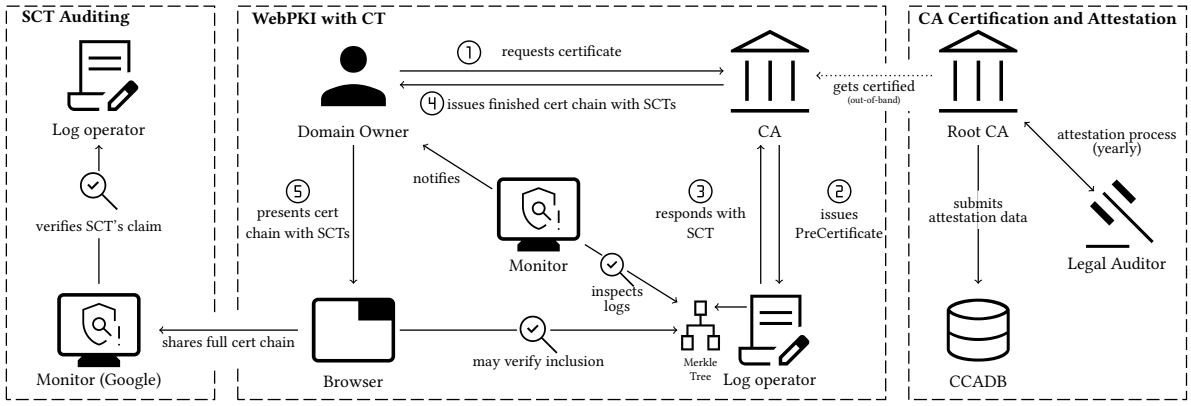


Figure 1: Different roles and usual information flow in the CT protocol with pre-certificates. The left box highlights the additional information flow for SCT auditing. The right box shows out-of-band steps of the WebPKI as outlined in the Baseline Requirements.

	Chrome [12], [13]	Edge	Firefox [14][15]	Safari [16]	Brave[3] [17]
CT supported / enforced	✓	✓[2]	✓	✓	✓
client-side inclusion check	✗	—	✗	✗	—
SCT Audit [13]	●[1]	—	✗	✗	—
# of required SCTs	2 [4]	—	2 [4]	2 [4]	2 [3,4]
# of trusted loggers	7	—	7	8	7–8 [3]

[1] for random set of certificates [2] no public policy, hence tested [3] follows policy of Apple or Chromium, depending on the platform
[4] 3 if validity > 180 days but only 2 from distinct log operators.

Table 1: CT features per Browsers according to their documentation and our testing. ● indicates that the feature is supported with some constraints. — shows that there was no information available.

and make the issuance of certificates transparent, Certificate Transparency (CT) introduces two new roles to the PKI ecosystem: logs and monitors [10].

Logs are responsible for maintaining a binary Merkle tree of all submitted certificates. The data structure can be efficiently checked for append-only and inclusion. Append-only means that a log between two commits to its Merkle tree, i.e., two *Signed Tree Heads* (STHs), can only ever add new certificates, but never remove or modify existing ones. The proof is calculated by presenting missing hashes of new entries that were added after the first STH. The old STH combined with these missing hashes should then yield the new STH. Inclusion means that a certificate is included in the log, which can be proven by providing the hashes of the siblings of the Merkle tree nodes on the path from the leaf to the root. Everyone can submit certificates to a log, but typically the signing CA does so by submitting a *pre-certificate*, a certificate whose sole purpose is to be included in a log but not yet accepted by other parties. Besides being a pre-certificate, it contains all the fields of the final certificate that will be issued later. On submission, the log replies with a *Signed Certificate Timestamp* (SCT). It contains the timestamp of submission and the log operator's identity. The SCT is signed where the signature is calculated over the SCT's content and the contents of the certificate the SCT is issued for. This allows one to verify the authenticity of an SCT and attribute it to a certificate. An SCT is a promise by the log to include the certificate together with its entire chain of CA certificates required to verify it in its Merkle tree within the *Maximum Merge Delay* (MMD), which is typically up to 24 hours. There are different ways to serve the SCTs to clients, but most commonly in practice, the CA includes the SCT in the final certificate [8], [19].

Monitors are responsible for continuous checking the logs in order to find rogue certificates. They fetch all certificates contained in a Merkle tree while also verifying the append-only property. Everyone can run a monitor or task a third party to do so. Third-party monitors scan logs for certificates that match the domain name of their customers to alert them [20]. The decision as to whether a certificate is rogue depends on the customers' knowledge. We present all roles in the PKI system with CT in Figure 1.

Known attacks and their detection. CT does not prevent malicious CAs from issuing rogue certificates, but it makes the issuance of such certificates transparent and thus detectable as long as the involved logs do not collude with the CA.

CT introduces new attack vectors for malicious logs, such as the partition attack, where a log can present different views of the Merkle tree to different parties [21]. Each party can verify the append-only property, but this only verifies consistency with party's previous views, not consistency with other parties's views. Equivocation is only detectable if different parties compare their

views of the log, a tactic known as *Gossiping* [22]. The omission attack violates transparency by issuing an SCT for a certificate without ever including it in the log. This attack is detectable by obtaining a failed inclusion proof together with an SCT that promised the opposite. Monitors that look for omitted rogue certificates will fail to detect the attack as long as they are not aware of the SCT that claimed something else. Relying parties that want to check for inclusion reveal which certificates they are interested in to the log. This makes it possible to infer the browsing history of the requesting party, which violates their privacy. [19], [23], [24].

SCT Auditing. To mitigate the omission attack, Google introduced SCT Auditing [24]. A dedicated monitor operated by Google receives SCTs from Chrome users who opted in and verifies their inclusion in the logs. It allows this particular monitor to *audit* the log against its issued SCTs. Figure 1 also shows the message flow for SCT Auditing.

Current deployment. Inspired by known weaknesses of CT, implementations found in browsers deviate from the standard and enforce their own policies. We investigated public design documents and tested the implementations in all major browsers in Table 1. All the policies we encountered define a small set of logs that are trusted and a number of SCTs that are required for a certificate to be accepted. All designs refer to CTv1 [10]. The newer version, CTv2 [25], has not been adopted by any implementation we are aware of, so we will focus on CTv1 in this work. Notably, no considered browser implements a client-side inclusion proof for privacy and efficiency reasons.

3 Background

3.1 Tamarin

Tamarin performs automated analysis in the Dolev-Yao model [26], wherein messages are described as abstract terms. Terms are composed from uninterpreted function symbols that represent cryptographic primitives and are applied to *variables* representing yet unknown values, as well as *names* serving as the base type, representing either high-entropy values like keys (then the name is *fresh*) or publicly known constants (then the name is *public*). If we want to clarify the type, we write $\sim n$ for the former and $\$n$ for the latter. The behavior of these cryptographic primitives is specified by an equational theory on terms. For example, a hash function is represented by a function symbol h and the empty equational theory, essentially describing it as a random oracle. We write $\mathsf{h}/1$ to indicate that h takes one parameter. Message signing and signature verification are typically written as the function symbols sign, verify, true and the equation $\mathsf{verify}(\mathsf{sign}(m, \mathsf{sk}), m, \mathsf{pk}(\mathsf{sk})) = \mathsf{true}$, meaning that verifying a correctly signed message reduces to the 0-ary function symbol (i.e., constant) true.

3.1.1 Multiset Rewriting (MSR). The overall state of a protocol execution is described as a multiset of *facts* where a fact has a fact symbol f and a list of terms, typically describing the state a protocol party is in. For example, $S_2(\$server, \sim nonce)$ if a server with id $\$server$ has previously received a nonce and is ready to respond.

Rules then describe the dynamics of the protocol. They have the form

$$l \; -[\; a \;]\!\rightarrow \; r$$

where the premises l contain a multiset of facts required to be in the current state for this rule to be applicable. These are removed and substituted with those in the conclusion r. A set of facts a, the actions, labels this transition.

Persistent facts, prefixed with !, will not be consumed. Such facts are, for instance, used to model the adversary's knowledge, which increases over time. The built-in facts In, Out and Fr model network input and output, and the choice of a fresh name, respectively.

3.1.2 Trace Properties. Given a set of rules, a protocol execution is any chain of rewrites starting from the empty multiset. A trace is the sequence of labels in an execution, but skipping empty labels $\emptyset$. Security properties are expressed in a first-order logic with quantification over terms and time points (indexes in the trace). The atom $f(\ldots)@i$ expresses a fact $f(\ldots)$ that occurs at time point i. The other atoms are term equality and comparison of time points.

3.2 Accountability

Tamarin has built-in support for formulating and proving accountability [27]. Their definition is protocol-agnostic and based on causality; a protocol provides accountability for property φ if it can always correctly identify the parties whose deviation from the protocol (jointly) caused [28] a violation $\neg\varphi$. Consequently, iff the protocol identifies no such set of parties, then φ must hold, hence accountability for φ implies verifiability for φ.

In our context, CT's task is to determine whether φ was violated and who was (part of) the cause for that. Practically, this means that CT specifies that each party provides proof for their own correct behavior and collects evidence of other parties' correct behavior. Parties might deviate but may be detected by other parties. There is no party with a complete view of the network, which distinguishes accountability in security protocols from accountability in distributed systems or software dependability. We consider the punishment of misbehavior to be outside the protocol. By contrast, the precise conditions under which a party is blamed we consider very much part of the protocol, and find currently under-specified. Hence, our task shall be to figure out these conditions (which we call *tests*) and the precise property φ.

To clarify who provides the inputs and what they indicate, we impose a naming convention for these events. $\mathtt{Assert}(id, c, s, p)$ means that the party with identity id makes a claim c regarding s w.r.t. a (possibly empty) list of parameters p. These events shall be emitted by rules modeling id's role in the protocol and be computable by these parties.

In Tamarin, tests are defined as trace properties with unbound variables for one or more party identities. For instance, the following tests check for a certificate that is later (externally) found to be incorrect.

```
test CAfalseAsserts_rootBlamed:
  "∃ intCA fields ca_fields #t0 #t1 #t2.
    Asserts('external_auditor',
            'That_intermediary_CA_signed_the_following_
 ↳ incorrect_statement_and_the_validation_chain_says_
 ↳ the_following_about_that_CA.',
            intCA, <fields, ca_fields>)@t0
```

```
∧ Asserts('CCADB', 'No_root_CA_signed_this_CA.',
          intCA, <ca_fields, rootCA>)@t1"
```

Observe that the underlined variable `rootCA` is unbound. These so-called *free* variables determine who the test blames. Given a trace t, this test blames *ca* if the test condition with the free variable instantiated to *ca* holds on t, written $t \vDash t\{\,\texttt{rootCA} \mapsto ca\,\}$. A test may contain multiple free variables, in which case these parties are *jointly* blamed. In a mixnet, for instance, an attack may necessarily involve multiple mixes working together. But a test can also apply multiple times on the same trace (e.g., if many attacks are mounted), or multiple tests may be defined and also apply on the same trace. In both cases, the instantiations of those tests assign blame sets that are *independently* blamed. Given a set of tests and a trace t, the *verdict* is the set of all sets of party identities for which some test is satisfied on t.

For instance, consider a second test below where the external auditor makes the same assertion, but the root CA correctly documented the intermediate CA, so the misbehavior must be assigned to the intermediate CA.

```
// [.. as in previous test ...]
∧ Asserts(rootCA, 'I_signed_this_intermediate_CA.',
          intCA, <ca_fields>)@t1
```

If both tests match, say, for *intCA* and *rootCA*, the blame set will be $\{\{\,intCA\,\},\{\,rootCA\,\}\}$, the responsibility is independent. Indeed, the CCADB, assumed honest, cannot make the conflicting assertions that the root CA has documented both the signing CA and has not, so these would be separate instances.

The verdict function induced by the set of traces should be correct, i.e., it should identify each minimal set of parties that (jointly) caused an attack (i.e., a violation of φ) and did so by deviating from their normal behavior (i.e., from the protocol). For instance, if t is such that three independent attacks happen, two matching the first test and one matching the second, then the verdict identifies three singleton causes, two of them root CAs and one intermediate CA. Correctness now asserts that for each attack, the respective intermediary or root CA was indeed deviating, and without them, φ would not have been violated (in that particular attack).

Morio and Künnemann provide us with a decomposition of accountability into trace properties, each necessary and all combined sufficient. This allows automated analysis with Tamarin, but the detailed decomposition is quite involved and not necessary for the understanding of the attacks we find. We present four of those trace properties that are intuitive and help us intuit attacks that we will see later.

- *sufficiency*: Each test has a trace that matches with only the blamed parties corrupted.
- *verifiability*: Should no test match, φ holds, and vice versa.
- *minimality:* No strict subset of some test's verdict can trigger that test or any other.
- *uniqueness:* Whenever a test matches, the blamed parties must be under adversarial control.

The remaining two (injectivity and single-matchedness) are never violated in this model and are less intuitive, hence we omit an explanation of them and instead refer the reader to [27]. Tamarin checks them via syntactic conditions.[1]

4 The Standard PKI

We start by laying out the PKI used on the web, which puts unverifiable trust into the CAs. In the first step we add an external validator and show that it can hold the previous parties accountable but needs itself to be trusted (by each client) and always be online. Although this is unrealistic, it helps transition to CT, which sets up infrastructure to distribute this validation task through the introduction of dedicated roles. In the following sections, we will proceed similarly for CT and extensions, adding new infrastructure (first monitors, then logs) and showing how trust moves from previously trusted roles to new, typically distributed, infrastructure. Each time, efficiency improves or parties have better incentives to remain honest.

We use standard function symbols $\mathrm{pk}/1$, $\mathrm{sign}/2$ and $\mathrm{verify}/3$ and the equation $\mathrm{verify}(\mathrm{sign}(x,y),x,\mathrm{pk}(y)) = \mathrm{true}()$ to model signatures. Certificates are signed 7-tuples representing the different fields of the certificate, including the identities of the issuer (1) and subject (2), the subject's public key (3), and the type (4). The remaining fields contain a fresh serial number (5) and a validity period, represented by two timestamps (6 and 7). In the equation above, x is this 7-tuple, and y is the issuer's secret key.

When the type is set to 'selfCert', it represents a trusted root certificate. Issuer and subject are the same; hence the certificate is signed with the public key that the tuple contains. When set to 'pubKey-CA' or 'pubKey', it is a regular public key certificate and signed with the issuer's public key, which is different from the subject's (intermediary CA's or end entity) public key. For 'pubKey-CA', the certificate can sign others, i.e., the subject is an intermediate CA. For 'pubKey', the certificate is a leaf, i.e., the subject is an end entity, e.g., a web server.

To give a flavor of modeling in Tamarin, we provide the multiset-rewriting (Section 3.1.1) rule that models the client validating an incoming certificate. Here, a CA that signed the certificate is already registered. To validate a certificate, the client checks the following:

(1) The certificate has been issued and signed by a CA that has been signed by a trusted root CA.
(2) The signature of the intermediate CA certificate verifies under the root CA's public key.
(3) The signature of the certificate verifies under the CA's public key.
(4) The certificate is still valid.

```
rule ClientValidate:
let
    ca_fields = <$rootCA,$CA,pkCA,'pubKey-CA',
                 snCA,$i0,$j0>
    fields = <$CA,$id,pkID,'pubKey',sn,$i,$j>
    chain = <<fields,pub_sig>,<ca_fields,ca_sig>>
in
[   In(chain), // incoming certificate
    !Root_CA($rootCA,pkRootCA), // check (1)
] -[ // check (2)+(3)
    Eq(verify(pub_sig,fields,pkCA),true()),
```

[1] The syntactic conditions are called BR and RP in [27]. For BR, all tests in this work succeed. For RP, the syntactic condition in Tamarin is too strict; we thus checked this condition manually, see Appendix C.

```
    Eq(verify(ca_sig,ca_fields,pkRootCA),true()),
    // check (4)
    CheckValidUntil($now,$j),
    CheckValidUntil($now,$j0),
    Time($now),
    // emitted event
    BrowserAcceptsChain($CA,fields,$rootCA,ca_fields),
]→ [ ]
```

The action `BrowserAcceptsChain` indicates that this rule was applied, i.e., a client accepted this certificate and the intermediate CA certificate as valid. We use it to state our security goal φ.

Modeling Timestamps. To perform check 4), we need to model the two timestamps included in certificates: time of creation and time of expiration. Tamarin does not have native support of time beyond ordering actions. We use the approach of Morio et al. [29] and model timestamps as public values that are assigned meaning by an axiomatic model. Using public variables ensures the adversary can access all timestamps, thereby also simplifying Tamarin's message deduction.

We annotate with an action `Time($now)` every rule that needs to measure time. We relate timestamps with the following restriction. Restrictions are properties that Tamarin assumes to hold.

```
restriction time_monotonicity:
"∀ t #t1 #t3. Time(t)@t1 ∧ Time(t)@t3 ∧ #t1 < #t3
⇒ (∀ tp #t2. Time(tp)@t2 ∧ #t1 < #t2 ∧ #t2 < #t3
      ⇒ tp = t )"
```

As monotonicity is sufficient for the properties we prove, we avoid other assumptions on timestamps. `Time` can occur in different parties; hence we assume a global clock.

We check whether an expiry date has passed by putting the action `CheckValidUntil` in the rule that performs the check. We then constrain traces to those where the check is evaluated correctly:

```
restriction certStillValid:
"∀ now valid_till #t1.
    Time(now)@t1 ∧ CheckValidUntil(now, valid_till)@t1
    ⇒ ¬(∃ #t2. #t2 < #t1 ∧ Time(valid_till)@t2)"
```

Accountable Authenticity. We posit that the main goal of the PKI is to provide an authentic mapping from identity to public key, or, put more generally, that when a browser accepts the information in a certificate `fields` from `CA`, then `CA` checked that this information is the ground truth, e.g., by validating the identity and ownership of a domain.

```
lemma authenticity:
"∀ CA fields #j.
   BrowserAcceptsChain(CA, fields, *, *)@j
   ⇒ ∃ #i. GroundTruth(CA, fields)@i ∧ #i < #j"
```

See Table 2 for the naming scheme we use for variables and Table 3 for the facts we will use as predicates in the lemmas. In particular, `BrowserAcceptsChain` will obtain more parameters when we introduce CT, so for uniformity, we use '*' to signify an arbitrary value, corresponding to all-quantified variables $i_1, i_2, \ldots$ that are otherwise ignored.

We immediately find that a corrupt CA can construct a rogue certificate (Lemma `authenticity` in `pki.spthy`) thereby refuting authenticity. Tamarin finds two possible corruption scenarios:

name	meaning
fields	content of certificate, a 7-tuple
ca_fields	like fields, but type field is 'pubKey-CA', typically information about the intermediary CA that signed a leaf certificate
ca_sig	signature over ca_fields, signed with root CA key
ca_cert	the tuple $\langle ca_fields, ca_sig \rangle$
CA	certificate authority (root or intermediary)
rootCA	root CA
L	logger
id_{log}	id of a log that is maintained by some logger
M	monitor
C	client
P	protocol participant of any role

Table 2: Commonly used variable names.

one where the signing CA itself is compromised, and another where the involved root CA misbehaves by signing a bogus CA certificate which is subsequently used to violate authenticity. A refined version that rules out these two scenarios holds (Lemma `cert_auth` in `pki.spthy`). Both corruption scenarios are possible; we need to be able to hold both individually accountable later.

To motivate CT, assume the user does not trust all root CAs and sometimes wants to validate whether a CA is still honest by externally checking the certificate, e.g., by calling the website owner on the telephone and comparing fingerprints. We thus add an external validation oracle that obtains a signed certificate and asserts whether its fields are correct or, with a second rule, that they are incorrect. With an additional restriction, we ensure that the validator never contradicts itself.

Now there is a middle ground between unconditional authenticity (refuted in Lemma `authenticity` in `pki.spthy`) and outright assuming the (root) CAs are honest (the aforementioned Lemma `cert_auth` in `pki.spthy`). We shall accept that authenticity may be violated, but hold the responsible CA accountable. Reconsider the tests from Section 3.2, where an external auditor finds incorrect fields within the certificate. As the certificate contains an intermediary CA's signature, the auditor can identify that intermediary CA as responsible (together with the relevant fields in the validation chain). Root CAs have to publish the intermediate CAs they signed in [30], [31], [32] into the publicly accessible CCADB database [18]. If the intermediate CA is not in the CCADB, a root CA is to blame, as they signed a rogue intermediate CA without documenting it. Otherwise, a trusted (by the root CA) intermediary CA misbehaved and caused the attack, so it is to blame. We can then prove the following lemma:

Listing 1: PKI provides acc. auth. assuming a trusted external validator.

```
lemma A_PKI_Ext:
 CAfalseAsserts_rootBlamed, CAfalseAsserts_rootNotBlamed
 accounts for
 "∀ CA rootCA ca_fields fields #t0 #t1.
 ExternalAuditorCheck(ca_fields, fields)@t0
```

name	model	meaning	parties providing evidence or assertion
BrowserAcceptsChain(CA,fields,rootCA,ca_fields,L_1,L_2)	all	Browser accepts leaf certificate with `fields` signed by `CA`, who's certificate is signed by `rootCA` and with SCTs from L_1 and L_2.	affected user
GroundTruth(CA,fields)	all	Honest domain owner (DO) provided evidence that fields are correct.	domain owner
ExternalAuditorCheck(ca_fields, fields)	PKI	External auditor completed checks for certification chain containing `ca_fields` and `fields`.	external auditor, CCADB
CTCheck(M, fields, CA, ca_fields)	CT	Monitor completed protocol-specific checks.	monitor, affected client (can be same), CCADB
SCTAuditCheck(M, fields, ca_fields)	SCTAudit		monitor (designated auditor), client, CCADB
SCTReceiptCheck(*M*, fields)	SCTAuditReceipt		monitor (designated auditor), client, CCADB
STHGossipCheck(*M*, *L*, fields, CA, ca_fields, s_1, s_2)	STHGossip		monitor, client, CCADB
CTTransparencyCheck(L, fields)	CT	Monitor completed protocol-specific transparency checks.	monitor
SCTAuditTr…(L, fields)	SCTAudit		monitor (designated auditor), client
STHGossipTr…(M, L, fields, CA, ca_fields, s_1, s_2)	STHGossip		monitor (designated auditor), client

Table 3: Facts that appear in Lemmas.

```
/*  if the certificate's statement is checked
externally... */
∧ BrowserAcceptsChain(CA, fields, rootCA, ca_fields)@t1
⇒
    (∃ #t. GroundTruth(CA, fields)@t)"
    // ...the certificate's statement is correct
```

We can use this lemma as a common template for the accountability properties of CT and its extensions.

Definition 4.1 (Accountable authenticity, informal). We say a model provides accountable authenticity for a check *C* (represented by a fact symbol) if the above lemma holds with *C* in place of `ExternalAuditorCheck`.

Note that this structure has an implicit liveness assumption: accountability holds only for those fields that were actually checked, i.e., the party performing the check was active. Typically, the rule emitting *C*(`ca_fields`, `fields`) is triggered by at least two rules, one indicating a violation of accountability and thus triggering some test, and another determining that the evidence for `ca_fields` and `fields` does not suggest misbehavior.

We observe that we have shifted trust from the CAs to external validation, which is inefficient. CT distributes this validation task to multiple monitors operating on a distributed log rather than relying on input from the client. Time to start exploring accountability in CT.

5 Certificate Transparency

Before we add loggers and monitors to the system, we have to amend certificates with a promise by a logger that it will add a certificate soon.

Signed Certificate Timestamps (SCTs). SCTs consist of three fields: the issuer (usually a logger), a type field, in our case just the type '`SCT`', and a timestamp to indicate when the SCT was issued. They represent the promise of the specified logger to add the certificate to the log by the issuing time plus the MMD (typically 24h). In our model, we modify the certificates to embed two SCTs from distinct logs, as all known browser policies require two SCTs (see Table 1). CT-conforming certificates have two embedded SCTs and carry the '`PubKey`' type. *Unfinished* public key certificates do not embed SCTs and have type '`pre-cert`', but are otherwise the same. The signature is calculated over all content fields of the certificate, including embedded SCTs.

Monitors and Loggers. We introduce loggers and monitors with initialization rules. As the monitor does not have its own key pair, its initialization rule only assigns a public variable, representing its identity. Loggers register a public key along with a self-signed certificate. Their key is later used to issue and sign SCTs. The main purpose of logs is to maintain the log structure and hand out commitments (STHs) and proofs for inspecting parties. We provide these rules in more detail in Appendix D.

In our model, domain owners can summon a monitor (either themselves or a third party, see Section 2) to check for new certificates whose subject matches the domain owner. We assume the domain owner provides the monitor some *ground truth* on the correctness of the fields in the certificate. Following the certification guidelines in CT, honest CAs validate the fields during pre-certification. We thus emit an event GroundTruth, which we can use to define rogue certificates (and, analogously, benign certificates) [10].

```
restriction rogueCert:
"∀ CA fields #t1.
 RogueCert(CA, fields)@t1 ⇒
 ¬(∃ #t0. GroundTruth(CA,fields)@t0 ∧ #t0 < #t1)"
```

Later, the monitor will make use of this knowledge to check for authenticity of certificates it encounters in the logs.

Certification. As discussed in Section 2, certification becomes a two-step process, which we model with two separate rules. The first issues a pre-certificate and sends it out on the network for two loggers to issue SCTs. The second receives these two SCTs and, knowing the pre-certificate, creates a public key certificate that embeds them. For the client, we adapt the rule `ClientValidate` from Section 4 to additionally verify the two embedded SCTs and add a condition asserting that the two loggers are different.

Modeling Logs. We model each logger's logs via trace properties, similar to how we modeled time. The action

$$\mathsf{LoggerComputesAddLog}(L, id_{log}, \ldots),$$

e.g., represents additions to the log id_{log} maintained by L and is given semantics *axiomatically*, through a restriction on traces. For the single-ledger case, earlier work [33], [34] followed the same

approach but could not represent inclusion proofs in messages on the network, as the log data structure was represented entirely on the trace. Explicit modeling of the underlying Merkle trees [35] does not scale as well (see Section 9 for discussion). Hence, we instead extend the axiomatic approach to handle multiple ledgers, MMD, inclusion proofs, and append-only proofs. Proofs need to be transmitted on the network and thus require an entirely new axiomatization. In particular, corrupt loggers must be able to produce 'incorrect' logs, so we have a reason to hold them accountable, but if they do, they cannot forge these proofs. Honest loggers also compute these proofs, but they never break these properties.

Instead of considering the log as the outcome of a series of computations, we record the computations themselves on the log. Each logger indexes a chain of computations on the logs with an identifier. When a logger L adds an entry f to a log l, it emits an action LoggerComputesAddLog(L, l, f) that indicates a correct addition to the Merkle tree via the function ADD [36]. We can model removal from the log by recomputing it from scratch with a fresh id, but honest loggers maintain only a single log id, as they adhere to CT. Dishonest parties, however, can compute multiple logs on the fly using all the information they know and thus perform an equivocation. Any computation that produces a valid Merkle tree is thus available; computations that produce invalid Merkle trees are only represented via malformed proofs, i.e., terms that do not pass validation. Merkle trees are never transmitted, only proofs about them; hence this is w.l.o.g.

We define predicates EntryVisible and EntryNotVisible, and use them in lemmas and restrictions to define the outcome of a sequence of such computations at a point in time.

```
EntryVisible(L, f, ca_cert, chain, id_log, #now, sth_ts) <=>
(∃ sct_ts #t.
/* f was added to L's log id_log at timepoint in SCT,
   which is the start of MMD period */
   LoggerComputesAddLog(L, id_log, f, ca_cert,
                        chain, sct_ts)@t
   ∧ #t < #now
/* ...MMD period has passed */
   ∧ ¬(sth_ts = sct_ts)
)
```

This rule takes the MMD into account by considering the time of the addition to the log and the time of the snapshot. As a simplification, we consider every action with the same `Time($i)` fact to belong to the same MMD period and every later timestamp to be after it. In this sense the addition becomes visible when that time has passed. EntryNotVisible is in Appendix D.

Inclusion proofs and snapshots. We represent inclusion proofs and commitments (STHs) as fresh variables, so they can be linked to the unique time point where they were computed. We use restrictions to link them to the log at that point in time (the *snapshot*) and give them meaning. The restrictions assert that iff a proof succeeds (i.e., the corresponding event is triggered on the nonce representing the proof) the proof statement holds true, i.e.,

- for an inclusion proof, the entry was visible (see predicate `EntryVisible`) at the time the proof was handed out.
- for an append-only proof, between two snapshots of a log, no entries disappeared.

Merkle trees and proofs constructed with them are thus assumed to work perfectly, which follows Dolev-Yao's 'perfect crypto' paradigm but encodes this assumption in the restriction instead of the term algebra.

Proof fetching is a three-step process. When a party, in our model either a client or a monitor, requests a proof, the logger chooses any snapshot that belongs to them (identified by the public variable id_{log}) and emits a fresh value representing the proof for that snapshot, i.e., the log at this moment. Attackers can present old snapshots of the log by producing a new log (with a new id_{log}) that contains the entries of the older snapshot. In the third step, the client or monitor receives this proof and checks for the property of interest. We add four restrictions to define the semantics of each proof type and for failure and success. We list and explain them in he full version Sections D.7 and D.8. Due to space limitations, we can only provide the restriction for a successful inclusion proof: for the branch marked with the event `InclusionProofSucceeds(..)` the logger indicated in the proof must have presented the view identified by id_{log} prior to the check and the entry whose inclusion is proven must satisfy the `EntryVisible` predicate for that time.

```
restriction inclusionProofWorks:
"∀ [...] .
 InclusionProofSucceeds(id_fetcher,L,fields,ca_cert,
                        chain, sth, session)@t5
 ⇒ ∃ id id_log time #t2.
  // Logger handed out a snapshot to the recipient..
  LoggerPresentsView(L,id,id_log,time,sth,session)@t2
  // ...prior to the check
  ∧ #t2 < #t5
  // and the entry is visible in that view
  ∧ EntryVisible(L,fields,ca_cert,chain,id_log,#t2,time)"
```

Note that we assume a private channel for this communication step. This is necessary because otherwise the network attacker could replay an old message from an honest log, producing a failing inclusion test that implicates the (honest!) log. This attack transfers to the real world, and indeed, CT demands the communication with the logger be implemented via HTTPS GET and POST requests. Any communication channel with a logger earns authenticity through hard-coded keys of the loggers. Otherwise, we would run into a chicken-and-egg problem, as the authenticity of the logger's public key using CT relies on the loggers themselves. The list of trusted loggers for instance, by Chrome [37] includes the public keys of the loggers to be used for pinning. The same applies to the list of trusted loggers by Apple and Mozilla [38], [39]. In practice monitoring tools such as CertSpotter rely on these public keys and use them to verify the signed statements by loggers, i.e., STHs [40].

Accountable Authenticity. As discussed before, we assume the monitor knows the ground truth. We thus add rules permitting it to raise an alarm, accept a correct certificate or misbehave by 'sleeping', i.e., receiving a certificate but not acting upon it. Each of these rules produces a fact *MonitorFoundCert* or *MonitorCouldNotFindCert* that is consumed by the actual CT check rules. These use the browser rules to determine which logs L_1 and L_2 should hold a certificate (the browser could just be run by the monitor itself) and check the result of the monitor for this certificate on L_1 and L_2. Again, this check may succeed or not, so there are two rules that perform it, each emitting the action *CTCheck*. The `CTcheck` rule for the case

where the monitor returns *MonitorCouldNotFindCert* waits until the MMD has passed and then additionally consults the CCADB so that the following test can determine whether to blame the intermediate CA or the root CA. Here, we display the test that blames any documented intermediate CA (CA) that issued a certificate which the monitor flagged as rogue.

```
test ca_auth_on_log:
"∃ M rootCA fields ca_fields #t0 #t1 #t2.
    CTCheck(M, fields, CA, ca_fields)@t0
    ∧ Asserts(M, 'Blame_this_CA.', CA, <fields,
      ca_fields>)@t1 // fields contradict ground truth
    ∧ Asserts(rootCA, 'I_signed_this_intermediate_CA.',
              CA, <ca_fields>)@t2"
```

The second test (omitted) does the same, but for undocumented intermediate CAs, thereby blaming any root CA that certified an undocumented intermediate CA.

Although we trust the monitor to confirm the certificate in question, we fail to hold CAs accountable. Tamarin provides a counterexample where a corrupt CA can issue a rogue certificate that is accepted by the client (Rule `ClientAcceptsCert_Key` in `ct.spthy`). Even if the honest monitor becomes active afterwards and fetches one of the logs that ought to contain the entry for that certificate, a corrupt logger can hide this entry from the monitor. The monitor would thus fail to blame the CA (let alone the log), and CT would not even ensure verifiability, i.e., that an attack leads to a nonempty blame set and thus at least one test matches. We can, however, show this lemma if we trust the logger.

Theorem 5.1 (Lemma `A_CT_1` in `ct.spthy`, Lemma `A_CT_2` in `ct.spthy`). *Against untrusted CAs, but with a trusted monitor and trusted logs, CTAuditCheck provides accountable authenticity. With only trust in the monitor, it does not.*

Transparency. Given that the log is honest, we have shown that we can prove accountable authenticity. This motivates our analysis of *transparency* as a separate property.

Ideally, we would demand that, whenever any client accepts a certificate, it can be found in both logs identified by the mandatory two SCTs. But when precisely is a certificate 'in a log'? Loggers are free to equivocate so they can maintain different logs, represented by different id_{log}, which they may present to different parties—or just keep to themselves, which would render this property useless.

Instead, the idea is that, once an entry has been presented as 'in the log', any party checking for this entry will find it. Again, we define a check, but also a fact marking its positive outcome. When a client accepts a certificate and someone checks for it later, that check must be successful. In the case of *accountable transparency*, if that check is not successful, the parties responsible for failure need to be identifiable.

Definition 5.2 (transparency, informal). A Tamarin model has a transparency check C if the occurrence of a fact $C(L, \texttt{fields})$ indicates that the check C has been conducted for a log L and a certificate statement `fields`. Let furthermore $C^{\top}$ indicate that the check was successful, i.e., the log L confirms at the time of the check to have that entry.

For a transparency check C, a model M provides *transparency* if the following lemma is valid

```
∀ CA fields rootCA ca_fields L1 L2 #t1 #t2.
 BrowserAcceptsChain(CA,fields,rootCA,ca_fields,L1,L2)@t1
  ∧ ( L = L1 ∨ L = L2)
  ∧ #t1 < #t2
  ∧ C(L,fields)@t2 ⇒ C⊤(L,fields)@t2
```

and *accountable transparency*, if a set of tests provides accountability of that property.

Transparency is also of independent interest to third parties that rely on the correctness of the logs, e.g., researchers [21]. Given CT's name, one could even argue that users might expect this property.

We show that transparency does not hold for dishonest loggers (Lemma `transparency` in `ct.spthy`). Assuming honest loggers, we find transparency to hold, but only after the MMD (Lemma `transparency_w_MMD_honest_log` in `ct.spthy`). Observing that honest loggers provide accountability, we can thus focus on the problem of holding the logger accountable.

6 SCT Auditing

Chrome adopted *SCT auditing* as an opt-in mechanism in [13], [24]. Instead of the clients, the monitors request inclusion proofs from the loggers. Clients upload a random selection of SCTs and certificate chains they have encountered to a particular version of the monitor currently maintained by Google. Having a large set of SCTs and underlying certificates at hand puts the monitor in a much stronger position: now, the monitor can validate a logger's entries and past STHs w.r.t. those. If the monitor knows the ground truth, it can use these certificates for authenticity checks. It can also enhance transparency by treating SCT auditing as a secondary source for discovering new certificates, rather than relying solely on logs. Finally, the received SCTs serve as promises that the monitor can verify. Any SCTs that do not have a counterpart in the log the monitor retrieves indicate misbehavior after the MMD has elapsed.

The downside to this proposal is that clients that opted in expose parts of their browsing history, as the certificates they emit contain the websites they visited as the subject.

Modeling. We add a rule for clients to report the certificates they encountered to a trusted monitor. As the mechanism is opt-in, the rule needs not be used. Accordingly, the properties below talk about the guarantees for participating clients, but consider a model where not every client does that. We add rules to the monitor that use incoming certificates and perform an authenticity check on them, emitting `SCTAuditCheck`.

As in the previous versions, the monitor relies on the ground truth it has received from the actual domain owner for this. There are two possible outcomes of this check, modeled in a rule each: Either the monitor finds a certificate inconsistent with its ground truth, or the certificate is authentic. Moreover, a dishonest monitor can find a rogue certificate but *sleep on it*, meaning they neither inform the subject nor blame the CA. It is necessary to distinguish this case from the monitor not receiving the certificate, as SCT audit relies on reception of the message, yet a misbehaving monitor can merely make a false claim (which we ignore in the model, as the domain owner can promptly verify it to be false) or ignore the received certificate.

Accountable Authenticity. We list the complete set of tests for accountable authenticity here. As before, the tests distinguish, based

on the CCADB, whether we blame the root CA or the intermediate CA, hence there are two. Both include the event `SCTAuditCheck`, thus precluding that the client (browser) opted into this check.

```
test ca_rogue_audit:
 "∃ M ca_fields fields rootCA #t #t0 #t1.
  SCTAuditCheck(M, fields, ca_fields)@t
  ∧ Asserts(M, 'This_CA_has_the_following_fields_and_it
 ↳ _failed_the_SCT_audit_for_the_following_fields.',
 ↳ CA,
            <fields, ca_fields>)@t0
  ∧ Asserts(rootCA, 'I_signed_this_intermediate_CA.',
 ↳ CA,
            <ca_fields>)@t1
 "

test root_ca_rogue_audit:
 "∃ M ca_fields fields CA #t #t0 #t1.
  SCTAuditCheck(M, fields, ca_fields)@t
  ∧ Asserts(M, 'This_CA_has_the_following_fields_and_it
 ↳ _failed_the_SCT_audit_for_the_following_fields.', CA
 ↳ ,
            <fields, ca_fields>)@t0
  ∧ Asserts('CCADB', 'No_root_CA_signed_this_CA.', CA,
            <ca_fields, rootCA>)@t1
 "
```

The monitor identifies misbehaving CAs correctly, even without trust in the loggers. Note, though, that the monitor is assumed to have ground truth.

Theorem 6.1 (Lemma `A_CTAudit_1` in `sct_audit.spthy`). *Against untrusted loggers and CAs, but with a trusted monitor, SCTAuditCheck provides accountable authenticity.*

If we do *not* trust the monitor, we find a counterexample where the monitor receives the rogue certificate but *sleeps on it* (ignores it), so the audit does not happen (Lemma `A_CTAudit_2` in `sct_audit.spthy`).

SCT Auditing mitigates the threat from malicious loggers hiding log entries. Having that *all clients* audit is a strong assumption, but a probabilistic check was actually deployed in Chrome. The above lemma is easily raised to the probabilistic setting, as it specifies its guarantees per audited certificate, e.g., if Chrome clients audit a certificate with probability p and n rogue certificates are sent to such clients, then an attack carries the risk of detection with a probability of $1 - (1 - p)^n$.

We also trust that the monitor performs the audit and knows the ground truth. This was a realistic assumption in previous scenarios, where companies could run their own monitors. Here, however, monitors receive certificates from all clients that have opted in, which is (a) a substantial load and (b) prohibitively slow for the clients if the monitor is not fast enough or has high latency. Finally, (c), if each server were trusted to also act as a monitor, then we would have eliminated the need for a PKI, let alone CT, altogether. It is thus not surprising that Chrome's monitor is run by Google and trusted by users running Google's web browser Chrome. Anyone can become a monitor, but not every monitor can perform SCT auditing.

Transparency. Uniquely for SCT auditing, the monitor can use the inclusion check to hold loggers accountable, too. This is useful for third parties that rely on the log. Chrome's design documentation hints that, indeed, such a check might exist.

> 'Google will [..] raise an alert if any log misbehavior is detected. The server-side auditing and alerting is covered in a separate Google-internal doc.' [13]

We find that transparency still fails (Lemma `transparency_SCT_Auditing` in `sct_audit.spthy`) but that we can provide accountable transparency, holding dishonest loggers accountable for failure to provide transparency.

```
test audit_inclusionViol:
"∃ M fields session #t #t1.
  Asserts(M, 'This_loggers_log_currently_does_not_
 ↳ include_these_fields.', L, <fields, session>)@t
∧ SCTAuditTransparencyCheckSession(L, fields, session)@t1
"
```

When the monitor performs an audit (which now requires it to obtain the STH externally, e.g., from a client), it fetches a log view and runs the usual inclusion check. If the monitor cannot find the certificate, the logger is blamed. With this procedure, we only need to trust the monitor, given that the client that provided the STH verified the inclusion of the certificate in that STH.

Theorem 6.2 (`T_Audit` in `sct_audit.spthy`). *Against untrusted loggers and CAs, but with a trusted monitor, SCTAuditTransparencyCheck provides accountable transparency.*

Holding the Monitor Accountable With Receipts. So far, we have excluded monitor misbehavior from our accountability analysis, as CT does not provide us with a mechanism suited for accountability tests. Other parties cannot verify whether the monitor purposefully ignores a rogue certificate or just did not receive it.

To see if that is possible at all, we extend SCT auditing, the only model managing to achieve accountable authenticity with a dishonest logger. We add a rule that has monitors hand over a signed receipt to the client, confirming the monitor has seen a particular snapshot of the log. Clients can require such a receipt before they proceed. In case of a dispute or suspicion, another rule has the client forward this receipt to the domain owner (alternatively, the domain owner could act as the client for testing).

In case a rogue certificate is in active use and found by the domain owner, an additional test can detect the monitor's misbehavior. While at that point, the domain owner could now just as well serve as an external validator like in section 4, the point here is that they can oversee the monitor service, which they contracted for this purpose. Moreover, SCT auditing works best if one monitor receives many SCTs and certificates, which would entail that this is an external service and independent enough that trust becomes an issue. Combined with the previous tests, we can now show accountable authenticity in the case where CAs, loggers, and monitors can be corrupted (Lemma `A_Receipt` in `ct_receipt.spthy`).

The model thus demonstrates a mechanism to hold monitors accountable as well. But note that it does not represent a published proposal, nor is it thought out at that level of detail.

7 STH Gossiping

We now turn to a different, yet similar approach to the equivocation problem, STH gossiping. While in SCT auditing, certificates had to be shared with the monitor to validate SCTs for the inclusion property, STHs can be validated for the append-only property without the certificate. This means STHs can be shared more widely:

Gossiping clients can share STHs with each other and verify append-only-ness between the snapshots they received and their own. To attack transparency, the logger must hide the entry from every gossiping client it interacts with, which prevents partition attacks on the gossiping clients. Direct communication is, however, impractical, since there are no standard communications method between browsers and end users are often difficult to address via IP because they are behind home routers or firewalls.

Who should the STH then be gossiped to? Nordberg et al. [22] presents various options, which can be classified by where the gossip is finally received and then analyzed. These are either the auditor or the monitor. We model specifically the case where STHs are sent to the monitor (comparable to *STH Pollination* [22]) or, equivalently, where the monitor doubles as a trusted auditor (*Trusted Auditor Relationship* [22]). Compared to *STH Pollination* (or *SCT Feedback* [22], which is the same but additionally for SCTs), we assume direct communication between client and monitor, instead of a relay via the web server, which was implemented as an HTTP header [21], [41] but is now deprecated. As we do not trust the web server, this abstraction comes without loss of generality. Our model thus abstracts both STH Pollination and Trusted Auditor Relationship, depending on whether we call the receiving monitor a 'trusted auditor' or not.

Modeling. We add a new rule that broadcasts the client's STH to the monitor. The monitor performs authenticity checks similar to those in CT, using the entries on the log to find potentially rogue certificates. In fact, we can reuse the same rules for authenticity checks in STH gossiping. What is new, however, is that the monitor receives a gossiped STH from a client. For this STH, inclusion of a specific entry may have already been proven by the client. To make use of this information, we add new rules to check append-only-ness between the gossiped STH and a freshly fetched snapshot from the log. So far, we have only considered append-only checks between snapshots that have been locally stored by the monitor or client, respectively. In this variant, one such STH is received from the network instead.

In Appendix F we provide the modified rules for this. They emit an assertion if the append-only check between the gossiped STH and the newly fetched snapshot fails. We assume the monitor to be honest and reuse its authenticity checks on the log from Section 5. If the certificate contradicts the known key for its subject, it blames the CA.

Accountable authenticity. The monitor fetches a new snapshot from the log in order to retrieve the same certificate and perform an authenticity check with it. If the certificate contradicts the ground truth, it will assert that the CA is rogue; otherwise, it proceeds silently. As before, we use this assertion and the previous legitimacy checks for intermediate CAs to decide to either blame the root CA or the intermediate CA.

Still, accountable authenticity does not hold. We find an attack against verifiability (Lemma `A_Gossip_1` in `sct_audit.spthy`). A malicious logger alone can bypass the test by first sending the client a correct certificate and an STH that proves inclusion, but then creating a new, incorrect, refutable inclusion proof for a different snapshot that does not contain this certificate. That proof will fail and the monitor will realize that something is wrong. It even correctly blames the logger, as discussed in the next section. But without the actual certificate, the monitor cannot decide if the CA is involved or not, or even whether authenticity was actually violated by whatever certificate the client accepted. Put differently, by creating an incorrect inclusion proof, the monitor can give rise to the impression that a certificate is missing and thus produce a false alarm, even if that certificate is completely benign. So, in contrast to SCT Auditing, sharing STHs with monitors is not sufficient to achieve accountable authenticity against untrusted loggers, but is sufficient for the monitor to detect equivocation. Gossiping ultimately achieves similar results to plain CT w.r.t. accountable authenticity, but with the added benefit of detecting equivocation by the logger (accountable transparency).

Theorem 7.1 (Lemmas `A_Gossip_0` and `A_Gossip_1` in `ct_gossip.spthy`). *Against untrusted CAs, but with trusted loggers and a trusted monitor,* STHGossipCheck *provides accountable authenticity.*

Transparency. The successful identification of the equivocating logger in the test for accountable authenticity gives us hope for (accountable) transparency.

Starting optimistically, we find that straightforward transparency is violated by a misbehaving logger (Lemma `transparency_Gossip` in `ct_gossip.spthy`). Assume that a client has a successful inclusion proof for a certificate in a given STH and gossips this STH with the monitor. The logger may now refuse to disclose the entries behind that fetched STH to the monitor and instead equivocate. This breaks transparency to the monitor, even though the client enjoys transparency.

However, a monitor can detect this situation if it additionally checks for the append-only property. We derive a test that achieves accountable transparency rather than transparency (Lemma `T_Gossip` in `ct_gossip.spthy`). Summarizing rules `GossipTransparencyCheck_Success` and `-_Failed`, the transparency check is run by the monitor for each target logger L. The monitor tests, for each gossiped STH it receives whether the most recent STH it fetched from the log is append-only w.r.t. to the gossiped STH and *assumes* that the client from which the gossip originates from has checked for inclusion—this is a modeling assumption stating that sufficiently many clients participate.

Theorem 7.2 (`transparency_Gossip` and `T_Gossip` in `ct_gossip.spthy`). *Against untrusted loggers and CAs, but with a trusted monitor,* STHGossipTransparencyCheck *provides accountable transparency, but does not provide transparency.*

8 Results

We summarize our results and provide details about our approach and the proof effort. All results were computed on an Intel 13th Gen Core i7-13700H with 16 allocated cores and 14 GB of memory. The proof files are available at https://github.com/rkunnema/formal-model-certificate-transparency; links to lemmas in this document are clickable and lead to anchors in an HTML document.

Sanity Lemmas. To show our models internally consistent, we show that each rule is reachable, thus ensuring that security statements are non-vacuous. We prove 192 such lemmas, one for each rule. Each is proven automatically, but we sometimes add restrictions to

protocol	authenticity with trust in …				transparency
	no one	monitor	+ logger	+ ext. validator	(no trust)
PKI	–	–	–	●	–
CT	✗	✗	●	● ⟹	✗
SCT Auditing	✗	●	● ⟹	● ⟹	●
+ receipts	●	● ⟹	● ⟹	● ⟹	● ⟹
Gossiping	✗	✗	●	● ⟹	●

(✗ = attack ● = acc. auth. / transp. ✓ = property proven – = not applicable)

Table 4: Results: Accountability and Transparency in CT. Click the icon to jump to the lemma in the browser. Accountable authenticity is implied by authenticity, so we additionally provide a counterexample against authenticity with the icon. A ⟹ denotes that a lemma follows logically (e.g., when adding more trust or for straight-forward extensions like adding receipts).

limit the search space (only for sanity lemmas and other lemmas that check for satisfiability, e.g., sufficiency). Their total verification time is less than two hours.

Protocol mechanics. To convince ourselves that we understand the mechanics of the protocol, we proved authenticity properties that found the trust between parties. Tamarin takes only a few minutes to prove all of them.

Lemma 1 (Lemma `cert_auth` in `pki.spthy`). *When a client accepts a certificate, then the CA asserts the value of this certificate unless this particular CA was corrupted.*

Lemma 2 (Lemma `cert_sct_auth` in `ct.spthy`). *If a CA asserts a public key certificate with two SCTs, then two distinct loggers have previously issued them honestly unless some party was corrupted.*

Lemma 3 (Lemma `sct_auth` in `ct.spthy`). *If a client accepts a certificate with two SCTs, then the two loggers they refer to have added an entry to their log unless one of them is corrupt.*

Accountability. We summarize the results discussed in the previous sections in Table 4. The overall verification time for all automated lemmas (sanity, accountability, transparency and helping lemmas) across all models is around five hours. For three helping lemmas and three accountability lemmas, we required partial manual intervention.

Transparency. Table 4 confirms the intuition that transparency is essential for achieving accountable authenticity—unless we can trust the logger. To hold CAs accountable, log entries must include the full certificate, including the fields signed by the CA. Without them, loggers may be able to accuse CAs wrongly. More generally, *what data* the protocol makes transparent determines who can be held accountable: full certificates enable accountability of CAs, SCTs suffice to hold loggers accountable for omissions. Observe that SCT auditing covers both. STH gossiping, however, reveals STHs to monitors, but not CA signatures, so CAs cannot be held accountable. This explains the gap: while STH gossiping can provide accountable transparency, which could be a building block for accountable authenticity, it fails to achieve accountable authenticity because it can only blame loggers correctly, but not CAs.

Applicability to Current Web Browsers. In Section 2, we discussed how different browsers implement CT and extensions. SCT auditing, implemented in Chrome, leads to strong transparency and (accountable) authenticity guarantees, but only if we assume every certificate is audited. In practice, not every Chrome client is configured to do this; those that are only audit only a randomized portion of certificates [13]. Nevertheless, we can state that every handshake that is actually audited enjoys these guarantees.

The other browsers neither implement SCT auditing nor provide client-side inclusion proofs. As we have seen, client-side inclusion proofs by themselves do not guarantee transparency—and thus accountability without trust in at least one logger—as a dishonest logger can equivocate. But they could be exploited to more effect. To avoid getting caught equivocating, the logger has to prove inclusion to clients but hide the entry from the monitor. Depending on the implementation, a logger can distinguish a client who requests an inclusion proof from a monitor who instead fetches all entries. If monitors mimicked the client's behavior by first fetching the STH and then requesting inclusion proofs for all known certificates, this distinction would be harder. We know existing implementations are not designed to do that, as clients are typically not communicating with the logger at all, except Chrome's designated SCT auditing monitor maintained by Google itself.

As of now, authenticity and transparency still rely on trust in the loggers. All browsers require $n = 2$ SCTs from distinct loggers. These can be different for each certificate, but must come from a pool of $N = 7$ or $N = 8$ trusted loggers (see Table 1). Assuming the attacker knows the user's browser, they can pick n dishonest loggers specifically. Hence, the trust assumption is that at least $N - (n-1)$ loggers are trusted (or at least, not colluding). Currently $N - (n-1)$ equals 6 or 7, depending on the browser. The logger pool is almost the same, so an attack would be largely independent of the browser.

9 Related Work

So far, formal analyses of CT have either been conducted manually, in the cryptographic standard model [42], or with automated tools in the Dolev-Yao model, but with significant simplifications [27], [35].

Cryptographic analyses of CT. Wrótniak et al. [42] modeled CT and conducted manual analysis to show transparency and accountability. They define accountability as a 23-line-long game in which the attacker wins if they are able to produce a rogue certificate that validates, although neither the root CA was corrupted nor any CA on the chain (including the root CA) was responsible for signing the rogue certificate's fields. They show that this notion of accountability holds for CT and PKIX.

First, this cryptographic game is not only specific to accountability for PKI schemes but also to their formulation, and encodes protocol-specific concepts like root certificates and root CAs, specific fields in certificates, and the data flow of the scheme. This definition is hard to adapt to formal analysis outside the cryptographic model, as it is game-based and very complex.

Secondly, because the game requirement is so specific, it is not clear whether it achieves accountability in a broader sense. Compared with the definition we employ, their definition at most guarantees that some responsible party can be identified. It does not ensure that this party indeed bears responsibility nor that everyone else who should bear responsibility is identified. Indeed, there is no mechanism to blame parties others than CAs or to blame more than one. Many works on accountability in protocol-like settings highlight the ability to blame (all) the responsible participants [43], [44], [45].

By contrast, our definition of accountability was tested on OCSP Stapling and Mixnets [27] and can ensure all parties blamed are causes of the violation and all such causes are indeed blamed. [45] employs protocol-specific policies to this end. For this reason, we can find that accountability only holds under strong assumptions. Nevertheless, [42] motivated the present analysis and helped formulate some tests (see below).

Automated Analysis of Accountability in CT. CT was first formally verified for accountability by Bruni et al. [33]. Morio and Künnemann [27] extend the model and analyze it with respect to the same accountability definition we use here, pointing out that their definition may miss violations and might blame a party for a violation of a security property that cannot be violated. We do not further extend their model but instead obtain a broader and much more realistic model that captures the details of the actual specification while continuing to support an unbounded number of participants.

Cheval et al. [35] analyzed CT (but no extensions) in ProVerif. Their focus is on accurately modeling the Merkle tree structure. They extend ProVerif to support an axiomatic modeling of ledgers. Besides CT, ledgers are also used in voting systems [46]. They first show the Merkle tree structure secure and then, assuming a Merkle tree interface, that a monitor receiving a proof can be sure that the certificate is in a (single, centralized) log, but not other properties. For instance, they model the proof of inclusion (they call this a proof of presence) using inductive clauses. They allow computing the proof recursively, starting from the leaf that contains the entry we are interested in, whose proof would be the empty list. Then, depending on whether the right or left child node is missing, the hash of that node is appended to the proof list together with a label `left` or `right`, indicating the branch taken to traverse the tree from the root to the leaf we started at. Our trace-based model of the log (Section 5) also allows for equivocation and represents proofs as messages that can be sent around but builds on restrictions that axiomatically assert the properties of the log in relation to inclusion or append-only proofs. We do not prove those axioms. Despite the more detailed modeling of Merkle trees, their model of CT is simpler than ours in arguably more pertinent aspects: they only consider browser-side validation (in a simplified manner) and browser-side monitoring (which is not implemented by any popular browser). Neither the loggers nor the monitors appear as parties distinct from the client or website and hence cannot be individually corrupted. In summary, for accountability, we had to design our model from scratch. Moreover, we are the first to model time to represent the certificate validity period and represent the pre-certification process.

SCT Auditing. Wrótniak et al. [42]'s model also considers SCT auditing [42, CTwAudit, Sec. D.2] (see Section 6). There, SCT auditing is given as an algorithm that, given a certificate with SCTs, returns the identity of the loggers that are deemed responsible for the lack of transparency should the corresponding entry not be visible in the log. They show that auditing can be used to achieve "audited transparency", i.e., either transparency holds or they are able to blame the responsible logger if transparency is violated. Similarly, we can show accountable transparency (for STH gossiping) or even transparency itself (for SCT auditing) under lighter assumptions. Our accountability tests for transparency in STH gossiping are similar to their algorithm for audited transparency, but they strictly focus on transparency with respect to the log. Instead of using the audited certificate to learn an entry, their algorithm 'magically' hands a certificate to the monitor, who has a snapshot from *every* log, and compares it with the existing knowledge. We do not assume that a monitor stores every certificate it has ever seen, but instead, it fetches another snapshot from a log and blames them if inclusion is violated.

10 Conclusion

We discussed the authenticity problem in PKIs that rely on a central authority. Our results support that CT solves this problem and successfully delegates the required trust from CAs to loggers. This helps; we gave the Symantec CA misbehavior as an example in the introduction. By logging the whole certificate and chain, log entries provide enough evidence to hold CAs accountable for the certificates they issued. Corrupt loggers can hide evidence, which constitutes a known attack against accountable authenticity and transparency. If one of the loggers in use is honest, transparency holds, but only for that particular logger. An honest monitor that knows the ground truth—every domain owner can be a monitor—can hold the CA accountable for maliciously issued certificates.

Assuming an honest logger is not as benign as it looks and requires further research from the policy perspective. The current pool of loggers consists of 7–8 loggers (see Table 1) with almost[2] complete intersect. Given Chrome's high market share, and the fact that certificates are transmitted before the user agent is known, there is no incentive for websites to include additional loggers, let alone exclude loggers from Chrome's list. Essentially, Google

[2] Currently, IPnG is included in some but not all.

decides which set of loggers is trusted, even for other browsers. Google's loggers are part of this set.

With SCT auditing, implemented in Chrome, we can remove the log from the trust assumption and instead hold misbehaving loggers and CAs simultaneously accountable. We find, however, that this strong guarantee only holds for certificates that are audited; for other certificates, loggers can get away with hiding the corresponding entry. Certificates are only audited randomly and only if the client uses Chrome and has opted in, thereby sharing them with one specific, trusted Google monitor. We discussed the privacy implications in Section 6. At the policy level, it is worthwhile to explore how to counteract the inherent tendency for centralization in SCT auditing.

With STH gossiping, which is deprecated and was not implemented, clients also share their log snapshot, but they additionally perform an inclusion proof with the corresponding logger. This adds further cost and effectively eliminates the privacy advantage gained from sharing only the log snapshot instead of full certificates. Future work could explore privacy-preserving primitives to compute inclusion proofs. Accountable transparency holds; while loggers can refuse to show the entries corresponding to an STH they have handed out, the monitor can identify this misbehavior. The same attack violates accountable authenticity.

Our model offers significant improvements compared to existing work, most notably the approach to Merkle proofs and its coverage of dishonest intermediary CAs, root CAs, loggers, and monitors. It currently does not cover certificate revocation. While CT does not oblige logs to store revocation requests, researchers are considering integrating revocation proofs into the CT log, e.g., via postcertificates [47] or similar entries [48] stored in the log that can prove a certificate was presented after being revoked. Vice versa, if revocation works, it also helps CT: if a monitor detects a rogue certificate, the most immediate reaction should be to revoke it. Accountability is as necessary for revocation as it is for certificate authenticity: Morio and Künnemann [27] showed that OCSP Stapling (RFC 6066) can provide accountability, but only if we trust the OCSP responder. Future work should investigate possible synergies between certificate transparency and revocation.

References

[1] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," Internet Engineering Task Force (IETF), Aug. 2018. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc8446

[2] "Baseline Requirements for the Issuance and Management of Publicly-Trusted TLS Server Certificates," *CA/Browser Forum*, May 16, 2025.

[3] "CA/Entrust Issues - MozillaWiki," Accessed: Mar. 16, 2026. [Online]. Available: https://wiki.mozilla.org/CA/Entrust_Issues

[4] J. Amann, O. Gasser, Q. Scheitle, L. Brent, G. Carle, and R. Holz, "Mission accomplished?: HTTPS security after diginotar," in *Proceedings of the 2017 Internet Measurement Conference*, London United Kingdom: ACM, Nov. 2017, pp. 325–340, ISBN: 978-1-4503-5118-8. DOI: 10.1145/3131365.3131401 Accessed: Mar. 16, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3131365.3131401

[5] Mozilla. "Revoking Trust in Two TurkTrust Certificates," Mozilla Security Blog, Accessed: Oct. 9, 2025. [Online]. Available: https://blog.mozilla.org/security/2013/01/03/revoking-trust-in-two-turktrust-certficates

[6] D. O'Brien, R. Sleevi, and A. Whalley. "Chrome's Plan to Distrust Symantec Certificates," Google Online Security Blog, Accessed: Jul. 15, 2025. [Online]. Available: https://security.googleblog.com/2017/09/chromes-plan-to-distrust-symantec.html

[7] R. Sleevi. "Sustaining Digital Certificate Security," Google Online Security Blog, Accessed: Jul. 15, 2025. [Online]. Available: https://security.googleblog.com/2015/10/sustaining-digital-certificate-security.html

[8] "Merkle Town," Accessed: Jul. 20, 2025. [Online]. Available: https://ct.cloudflare.com/

[9] B. Laurie, "Certificate transparency," *Communications of the ACM*, vol. 57, no. 10, pp. 40–46, Sep. 23, 2014, ISSN: 0001-0782, 1557-7317. DOI: 10.1145/2659897 Accessed: Mar. 16, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/2659897

[10] B. Laurie, A. Langley, and E. Kasper, "Certificate Transparency," RFC Editor, 6962, Jun. 2013, RFC 6962. DOI: 10.17487/RFC6962 [Online]. Available: https://www.rfc-editor.org/info/rfc6962

[11] "User Agents : Certificate Transparency," Accessed: Jul. 3, 2025. [Online]. Available: https://certificate.transparency.dev/useragents/

[12] "Chrome Certificate Transparency Policy," CertificateTransparency, Accessed: May 27, 2025. [Online]. Available: https://googlechrome.github.io/CertificateTransparency/ct_policy.html

[13] E. Stark and C. Thompson. "Opt-in SCT Auditing (public)," Google Docs, Accessed: Mar. 17, 2026. [Online]. Available: https://docs.google.com/document/d/1G1Jy8LJgSqJ-B673GnTYIG4b7XRw2ZLtvvSlrqFcl4A

[14] "SecurityEngineering/Certificate Transparency - MozillaWiki," Accessed: Apr. 2, 2025. [Online]. Available: https://wiki.mozilla.org/SecurityEngineering/Certificate_Transparency

[15] "Firefox," Firefox for Android 145.0 Release Notes, Accessed: Apr. 18, 2026. [Online]. Available: https://www.firefox.com/en-US/firefox/android/145.0/releasenotes/

[16] "Apple's Certificate Transparency policy," Apple Support, Accessed: May 27, 2025. [Online]. Available: https://support.apple.com/en-us/103214

[17] "TLS Policy," GitHub, Accessed: May 27, 2025. [Online]. Available: https://github.com/brave/brave-browser/wiki/TLS-Policy

[18] "Common CA Database by the Linux Foundation," Accessed: Oct. 14, 2025. [Online]. Available: https://www.ccadb.org/policy#5-audit-disclosures

[19] N. Blagov, "State of the Certificate Transparency Ecosystem," 2020. DOI: 10.2313/NET-2020-11-1_09 Accessed: Jun. 30, 2025. [Online]. Available: https://www.net.in.tum.de/fileadmin/TUM/NET/NET-2020-11-1/NET-2020-11-1_09.pdf

[20] "Monitors : Certificate Transparency," Accessed: Jul. 9, 2025. [Online]. Available: https://certificate.transparency.dev/monitors/

[21] O. Gasser, B. Hof, M. Helm, M. Korczynski, R. Holz, and G. Carle, "In log we trust: Revealing poor security practices with certificate transparency logs and internet measurements," in *Passive and Active Measurement*, R. Beverly, G. Smaragdakis, and A. Feldmann, Eds., Cham: Springer International Publishing, 2018, pp. 173–185, ISBN: 978-3-319-76481-8.

[22] L. Nordberg, D. K. Gillmor, and T. Ritter, "Gossiping in CT," Internet Engineering Task Force, Internet Draft (Expired) draft-ietf-trans-gossip-05, Jan. 14, 2018, 57 pp. Accessed: Apr. 2, 2025. [Online]. Available: https://datatracker.ietf.org/doc/draft-ietf-trans-gossip

[23] S. Meiklejohn, J. DeBlasio, D. O'Brien, C. Thompson, K. Yeo, and E. Stark, "SoK: SCT Auditing in Certificate Transparency," *Proceedings on Privacy Enhancing Technologies*, vol. 2022, no. 3, pp. 336–353, Jul. 2022, ISSN: 2299-0984. DOI: 10.56553/popets-2022-0075 Accessed: Mar. 18, 2025. [Online]. Available: https://petsymposium.org/popets/2022/popets-2022-0075.php

[24] "SCT Auditing Google Source," Google. [Online]. Available: https://chromium.googlesource.com/chromium/src/+/refs/heads/main/services/network/sct_auditing/

[25] B. Laurie, E. Messeri, and R. Stradling, "Certificate Transparency Version 2.0," RFC Editor, 9162, Dec. 2021, RFC9162. DOI: 10.17487/RFC9162 [Online]. Available: https://www.rfc-editor.org/info/rfc9162

[26] S. Meier, B. Schmidt, C. Cremers, and D. Basin, "The TAMARIN Prover for the Symbolic Analysis of Security Protocols," in *Computer Aided Verification*, N. Sharygina and H. Veith, Eds., red. by D. Hutchison et al., vol. 8044, Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 696–701, ISBN: 978-3-642-39798-1 978-3-642-39799-8. DOI: 10.1007/978-3-642-39799-8_48 Accessed: Jul. 2, 2025. [Online]. Available: http://link.springer.com/10.1007/978-3-642-39799-8_48

[27] K. Morio and R. Künnemann, "Verifying Accountability for Unbounded Sets of Participants," in *2021 IEEE 34th Computer Security Foundations Symposium (CSF)*, Dubrovnik, Croatia: IEEE, Jun. 2021, pp. 1–16, ISBN: 978-1-7281-7607-9. DOI: 10.1109/CSF51468.2021.00032 Accessed: Mar. 18, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/9505190/

[28] J. Y. Halpern. "A Modification of the Halpern-Pearl Definition of Causality." arXiv: 1505.00162 [cs], Accessed: Jul. 10, 2025. [Online]. Available: http://arxiv.org/abs/1505.00162 pre-published.

[29] K. Morio, I. Esiyok, D. Jackson, and R. Künnemann, "Automated Security Analysis of Exposure Notification Systems," in *32nd USENIX Security Symposium (USENIX Security 23)*, USENIX Association, Aug. 2023, pp. 6593–6610. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/morio

[30] "Chrome Root Program Policy, Version 1.7," Accessed: Nov. 2, 2025. [Online]. Available: https://googlechrome.github.io/chromerootprogram/#2-common-ca-database

[31] "Root Certificate Program - Apple," Accessed: Nov. 2, 2025. [Online]. Available: https://www.apple.com/certificateauthority/ca_program.html

[32] "Mozilla Root Store Policy," Mozilla, Accessed: Nov. 2, 2025. [Online]. Available: https://www.mozilla.org/en-US/about/governance/policies/security-group/certs/policy/

[33] A. Bruni, R. Giustolisi, and C. Schuermann, "Automated Analysis of Accountability," in *Information Security*, P. Q. Nguyen and J. Zhou, Eds., vol. 10599, Cham: Springer International Publishing, 2017, pp. 417–434, ISBN: 978-3-319-69658-4 978-3-319-69659-1. DOI: 10.1007/978-3-319-69659-1_23 Accessed: Apr. 2, 2025. [Online]. Available: https://link.springer.com/10.1007/978-3-319-69659-1_23

[34] R. Künnemann, I. Esiyok, and M. Backes. "Automated Verification of Accountability in Security Protocols." arXiv: 1805.10891 [cs], Accessed: Mar. 18, 2025. [Online]. Available: http://arxiv.org/abs/1805.10891 pre-published.

[35] V. Cheval, J. Moreira, and M. Ryan. "Automatic verification of transparency protocols (extended version)." arXiv: 2303.04500 [cs], Accessed: Mar. 17, 2025. [Online]. Available: http://arxiv.org/abs/2303.04500 pre-published.

[36] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in *Proceedings of the 18th Conference on USENIX Security Symposium*, ser. SSYM'09, Montreal, Canada: USENIX Association, 2009, pp. 317–334.

[37] Chrome. "Known CT logs," Accessed: Aug. 2, 2026. [Online]. Available: https://www.gstatic.com/ct/log_list/v3/log_list.json

[38] Apple. "Current CT Log List," Accessed: Aug. 2, 2026. [Online]. Available: https://valid.apple.com/ct/log_list/current_log_list.json

[39] Mozilla. "Known CT Logs," Accessed: Aug. 2, 2026. [Online]. Available: https://hg-edge.mozilla.org/mozilla-central/file/tipsecurity/ct/CTKnownLogs.h

[40] SSLMate. "Certspotter/monitor/monitor.go at master · SSLMate/certspotter," GitHub, Accessed: Aug. 14, 2026. [Online]. Available: https://github.com/SSLMate/certspotter/blob/master/monitor/monitor.go

[41] "Expect-CT header - HTTP | MDN," Accessed: Jun. 30, 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Expect-CT

[42] S. Wrótniak, H. Leibowitz, E. Syta, and A. Herzberg, "Provable Security for PKI Schemes," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, Salt Lake City UT USA: ACM, Dec. 2, 2024, pp. 1552–1566, ISBN: 979-8-4007-0636-3. DOI: 10.1145/3658644.3670374 Accessed: Mar. 19, 2025. [Online]. Available: https://dl.acm.org/doi/10.1145/3658644.3670374

[43] J. Feigenbaum, A. D. Jaggard, and R. N. Wright, "Open vs. closed systems for accountability," in *Proceedings of the 2014 Symposium and Bootcamp on the Science of Security*, Raleigh North Carolina USA: ACM, Apr. 8, 2014, pp. 1–11, ISBN: 978-1-4503-2907-1. DOI: 10.1145/2600176.2600179 Accessed: Apr. 4, 2025. [Online]. Available: https://dl.acm.org/doi/10.1145/2600176.2600179

[44] H. Chockler and J. Y. Halpern. "Responsibility and blame: A structural-model approach." arXiv: cs/0312038, Accessed:

Apr. 8, 2025. [Online]. Available: http://arxiv.org/abs/cs/0312038 pre-published.

[45] R. Küsters, T. Truderung, and A. Vogt, "Accountability: Definition and relationship to verifiability," in *Proceedings of the 17th ACM Conference on Computer and Communications Security*, Chicago Illinois USA: ACM, Oct. 4, 2010, pp. 526–535, ISBN: 978-1-4503-0245-6. DOI: 10.1145/1866307.1866366 Accessed: Apr. 8, 2025. [Online]. Available: https://dl.acm.org/doi/10.1145/1866307.1866366

[46] V. Cortier, J. Lallemand, and B. Warinschi, "Fifty Shades of Ballot Privacy: Privacy against a Malicious Board," in *2020 IEEE 33rd Computer Security Foundations Symposium (CSF)*, Jun. 2020, pp. 17–32. DOI: 10.1109/CSF49147.2020.00010 Accessed: Jul. 10, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/9155128/

[47] N. Korzhitskii, M. Nemec, and N. Carlsson. "Postcertificates for Revocation Transparency." arXiv: 2203.02280 [cs], Accessed: Oct. 13, 2025. [Online]. Available: http://arxiv.org/abs/2203.02280 pre-published.

[48] J. Kong, D. James, H. Leibowitz, E. Syta, and A. Herzberg. "CTng: Secure Certificate and Revocation Transparency," Accessed: Oct. 13, 2025. [Online]. Available: https://eprint.iacr.org/2021/818 pre-published.

[49] T. Treitz and R. Künnemann, *Accountability in certificate transparency and variants*. [Online]. Available: https://github.com/rkunnema/formal-model-certificate-transparency/blob/main/README.md

A Ethics considerations

We perform a defense-focussed analysis on the specification level. No experiments were conducted on living subjects.

The potential impact of this work is to provide clarity over the existing guarantees that CT provides, and what structural changes can improve those. Moreover, new suggestions that may improve the transparency and accountability properties, but may incur a little network overhead. Attacks we find pertain to protocols whose accountability guarantees were so far not clearly communicated, so they should be considered boundary conditions of our positive results.

B Open Science

We provide in the supplementary material [49]:

- the model files
- manually derived proofs
- a reproducible environment based on Nix flakes to rerun the experiments, including a fixed version of tamarin and batch-tamarin for convenient display of results
- recorded proof and evaluation results for the reviewer's convenience
- marked up html version of the model files that the macros in this file refer to
- a `README.md` file that explains how the material is organized and how the experiments are reproduced.

We plan to submit these model files to the Tamarin main repository eventually as well as to a long-term archival website.

C Replacement Property

For every accountability lemma the Replacement Property (RP) must be manually verified when the model uses public constants or restrictions. Because the model uses both extensively, we provide a brief argument that RP is not violated in our model. The property states that for every single-matched trace of a case test, the instantiation of free parties can be replaced by any other party which is permitted by our model. Our model uses public constants as type identifiers for different certificates. By design, these public constants are not used as identifiers for different roles and for each role initialization, we restrict that the used public variable is unique among different roles. We need to argue that all used restrictions preserve the replacement property. All inlined restrictions are concerned with asserting function symbol outcomes, distinctness of SCTs and the loggers involved in them. Two more inline restrictions state that the two loggers considered in SCTs must be distinct. They remain distinct under bijective renaming. All restrictions that axiomatically assert the Merkle proofs and content delivered by a logger are independent of role identifiers, and thus they do not restrict the identities of involved parties. One additional restriction determines whether the CA documentation is valid for any party role and whether a certificate is valid or benign in an external check. We restrict CAs, loggers and domain owners to register only once to ensure uniqueness of identities such that the public variable cannot be reused for different roles, including monitors. These eight restrictions remain distinct under bijective renaming. Three more restrictions concern the uniqueness of log identifiers. Recall that these log identifiers are distinct from the identifiers of loggers. The remaining restrictions concern equality, time axioms, and consistent assertions used in the external check. None of them influence role identifiers. We use two more restrictions in the additional receipt model to decide whether a monitor is blamed. This holds for any monitor.

Thus our restrictions and use of public constants do not violate RP and our analysis results are valid.

D Modeling CT (Details)

D.1 Adding Loggers and Monitors

We introduce loggers and monitors similarly to the other roles. Both roles have an initialization rule. As the monitor does not have its own key pair, it only consists of a public variable representing its identity. Loggers register a public key that is later used to issue and sign SCTs. The main purpose of logs is to maintain the log structure and hand out commitments and proofs for inspecting parties. The log model is described in detail are given in Section D.3.

```
rule Monitor_Init:
 [  ]
 -[ Time($now), Monitor_Registered($M) ]->
 [ !Monitor($M) ]

rule Logger_Init:
 [ Fr(~skL) ]
 -[ Time($now), Logger_Registered($L) ]->
 [ !Logger_s($L, ~skL, pk(~skL)), !Logger($L, pk(~skL)) ]
```

We model the monitoring check by adding a rule that represents a monitor starting to observe a logger and a second rule that represents the start of tracking a specific subject.

```
rule StartMonitoring:
 [ !Monitor($M), !Logger($L, pkL) ]
 -[ Time($now) ]->
 [ !LoggerDict($M, $L) ]

rule monitorTrackSubject:
 [ !Monitor($M), !DomainOwner(su, pubK) ]
 -[ Time($now) ]->
 [ !TrackSubject($M, su) ]
```

D.2 Modified Certification

```
rule PreCertify: /* CA issues a pre-certificate */
let
 preC_fields =
    <$CA, $id, pkID, 'pre-cert', ~sn, $now, $j>
 preC_sig = sign(preC_fields, skCA)
 preC = <preC_fields, preC_sig>
in
 [
  !DomainOwner($id, pkID),
  !CA_s0($CA, skCA, pkCA, ca_fields, ca_sig), Fr(~sn) ]
 -[
  GroundTruth($CA,
              <$CA, $id, pkID, 'pubKey', ~sn, $now, $j>),
  Time($now)
 ]->
 [ Certificate_Store($CA, preC), Out(preC_fields),
   Out(preC_sig), Out(ca_fields), Out(ca_sig) ]

rule PubKeyCertify:
let
 preC_fields =
    <$CA, $id, pkID, 'pre-cert', sn, $i, $j>
 sct1_fields =
    <$L1, 'SCT', $sct1, <$CA, $id, pkID, sn, $i, $j>>
 sct2_fields =
    <$L2, 'SCT', $sct2, <$CA, $id, pkID, sn, $i, $j>>
 sct1 = <$L1, 'SCT', $sct1, sct1_sig>
 sct2 = <$L2, 'SCT', $sct2, sct2_sig>
 pub_fields =
    <$CA, $id, pkID, 'pubKey', sn, $i, $j, sct1, sct2>
 fields = <$CA, $id, pkID, 'pubKey', sn, $i, $j>
 pub_sig = sign(pub_fields, skCA)
in
[ !CA_s0($CA, skCA, pkCA, ca_fields, CA_sig),
  /* 2 SCTs: */  In(sct1), In(sct2),
  !Logger($L1, pkL1), !Logger($L2, pkL2),
  Certificate_Store($CA, <preC_fields, sig_preC>)
]
-[
  Eq(verify(sct1_sig, sct1_fields, pkL1), true()),
  Eq(verify(sct2_sig, sct2_fields, pkL2), true()),
  Eq(verify(sig_preC, preC_fields, pkCA), true()),
  _restrict( ¬($L1 = $L2) ),
  CheckValidUntil($now, $j),
  Time($now)
]->
[ Out(pub_fields), Out(pub_sig) ]
```

D.3 Log Data Structure

Using the trace as ledger. We model logs maintained by every logger exclusively as a property of the trace. We add actions to rules to make additions to the log, as well as the generation and verification of proofs visible. Then we use trace restrictions to enforce relations between these. This is similar to the model of time and space in [29]. For example, the action `LoggerComputesAddLog ↳ ($LogIdentity, ...)` represents additions to the log maintained by `$LogIdentity`.

A similar approach has been used before by Bruni et al. [33] and extended by Künnemann et al. [34] to model CT's public ledgers. Both capture the partition attack by allowing the logger to present different snapshots to different auditors. Inclusion or append-only proofs were not part of either models, and we show that the approach can be further extended to abstractly capture properties of the Merkle tree structure, including append-only proofs and inclusion proofs.

An honest logger accepts an incoming pre-certificate only if it has a full chain that ends with a known root certificate. There are different notions in literature whether the root certificate itself is considered a part of the chain. In our model, the root CA certificate is not part of the chain as every participant in the system has access to it anyway. Then, it issues the corresponding SCT and adds the chain and pre-certificate to its publicly visible log. Following [10], the entry in the Merkle tree later (which we refer to interchangeably as log) only contains the pre-certificate without the signature. Note that this has practical reasons, as later with inclusion proofs, the requesting party needs to specify which entry it wants to receive a proof of inclusion. The party might not have the full chain available, nor is it guaranteed that the exact same chain is stored in the log (think of corruption cases). The promise by the log that clients receive as SCT is only computed over the pre-certificate fields and not a chain. Thus we can only hold a logger liable for the pre-certificate fields and not the complete chain. However, [10] also specifies that loggers MUST store the full chain for auditing purposes. Our action fact `LoggerComputesAddLog`, thus contains both the pre-certificate fields and the full chain. Inclusion proofs are later formalized over the pre-certificate fields while monitors inspecting the full log will also get access to the full chain.

```
rule add_preC_to_log:
let
 preC_fields = <$CA, $id, pkID, 'pre-cert', sn, $i, $j>
 ca_fields =
    <$rootCA, $CA, pkCA, 'pubKey-CA', snCA, $i0, $j0>
 fields = <$CA, $id, pkID, 'pubKey', sn, $i, $j>
 sct_fields =
    <$L, 'SCT', $now, <$CA, $id, pkID, sn, $i, $j>>
 sct_sig = sign(sct_fields, skL)
 sct = <$L, 'SCT', $now, sct_sig>
 chain = <<preC_fields, preC_sig>, <ca_fields, ca_sig>>
in
[
  !Root_CA($rootCA, pkRootCA), !Log($L, id_log),
  /* Hardcoded key material of the root CA */
  !Logger_s($L, skL, pk(skL)),
  /* Logger key material (known to relying parties) */
  In(preC_fields), In(preC_sig),
  In(ca_fields), In(ca_sig)
  /* incoming pre-cert chain */
]
-[
  Eq(verify(preC_sig, preC_fields, pkCA), true()),
    /* honest additions have to be properly signed */
  Eq(verify(ca_sig, ca_fields, pkRootCA), true()),
  LoggerComputesAddLogFields($L, id_log, fields, $now),
  LoggerComputesAddLog($L, id_log, fields,
                       <ca_fields, ca_sig>, chain, $now),
```

```
  LoggerAsserts($L, sct),

  CheckValidUntil($now, $j),
  Time($now),
⊢
[ Out(sct) ]
```

D.4 Corrupted Loggers

Like for CAs, we add a key leakage rule for the logger. This allows the network attacker to construct forged SCTs. To allow manipulation of the log entries, we add rules that can only be used by corrupted loggers.

A maintained log provides three properties that can be checked by every participant:

(1) Append-only: No entries disappear from the log over time.
(2) SCT Inclusion: Issued SCTs are backed by an entry in the log.
(3) Entry Validity: Every entry in the log corresponds to a valid pre-certificate with a full chain ending in a trusted root.

We add the ability to break precisely these properties in any possible way. Every logger can maintain arbitrarily many logs; each log can be described by the operations of addition and proof construction.

This snapshot is obtained by adding to every operation on the log (addition, proof construction) a id_{log}, representing the log on which the action is computed. Whenever some party inspects the entries, the logger controls which log it represents. Honest loggers maintain just one log, i.e., there is an injection from `$IdL` to id_{log}.

```
rule Start_Log_malicious:
 [ !Corrupted_Logger($L, ~skL, pk(~skL)) ]
 -[ StartedLog($L, id_log), Time($now) ]->
 [  ]

rule forge_log_view:
let
 preC_fields = <$CA, $id, pkID, 'pre-cert', sn, $i, $j>
 fields = <$CA, $id, pkID, 'pubKey', sn, $i, $j>
 ca_fields =
    <$rootCA, $CA, pkCA, 'pubKey-CA', snCA, $i0, $j0>
 chain = <<preC_fields, preC_sig>, <ca_fields, ca_sig>>
in
[ !Corrupted_Logger($L, skL, pk(skL)), In(preC_fields),
  In(preC_sig), In(ca_fields), In(ca_sig),
  /* Incoming certificate that is added */ ]
-[
  /* observe that we can have arbitrary forged additions:
  there is no requirement for proper signing */
  LoggerComputesAddLogFields($L, id_log, fields, $now),
  LoggerComputesAddLog($L, id_log, fields,
                       <ca_fields, ca_sig>, chain, $now),
  Time($now)
]->
[ ]
```

The forge log entry rule allows corrupted loggers to add arbitrary entry to any of their logs without possessing a valid pre-certificate chain.

D.5 Visibility Predicates and Maximum Merge Delay

Maximum Merge Delay (MMD). In CT, loggers are allowed to have a time period between SCT issuance and the corresponding entry becoming visible in the log, called maximum merge delay (MMD). We include this notion in our model by modifying additions to the log and changing the visibility predicates. Additions to the log now become scheduled at the time they are performed. The fact is annotated with the current model time according to the `Time` ↳ action fact. Scheduled additions are pending and, in the sense of MMD, become visible to inspecting parties after the delay has passed.

Visibility of Merkle Tree Leaves. To formulate that an entry is visible at a given time point in the trace, we introduce visibility predicates that can be reused for this purpose. Intuitively we say that an entry is visible in the considered log with id_{log} if at the given time point on the trace, the entry has been added (`LoggerComputesAddLog`) before. Likewise, an entry is not visible if it has never been added. We formalize visibility as predicates based on the `#time` of snapshot creation.

```
EntryVisible(L, fields, ca_cert, chain, id_log, #time,
             sth_time) <=>
 (∃ different_time #t.
  LoggerComputesAddLog(L, id_log, fields, ca_cert,
                       chain, different_time)@t
    /* there has been an addition to id_log with the
       fields and chain... */
  ∧ #t < #time
    /* ...prior to #time and... */
  ∧ ¬(sth_time = different_time)
    /* ...that addition was visible */
 )

EntryMissing(L, fields, id_log, #time, sth_time) <=>
 ¬(∃ ca_cert different_time chain #t.
    LoggerComputesAddLog(L, id_log, fields, ca_cert,
                         chain, different_time)@t
    ∧ ¬(sth_time = different_time)
     /* or there has never been a visible addition
        to the log */
    ∧ #t < #time
    )
```

We include the time of the proof handout (`proof_time`) in the predicate. Additions to the log are visible at `proof_time`, if they have been scheduled to a strictly `different_time`, which represents a different MMD interval. Removals from the log are assumed to become immediately visible when they occur in the trace.

`EntryNotVisible` is adapted similarly: either the entry has not been *visibly* added before, or it has been visibly added but removed after and not visibly added after the removal at the time of the inspection.

In the following, we will assume these modified visibility predicates and the setting with MMD.

Note here that these predicates are not concerned with the chain. The visibility predicate is exclusively a property defined on the Merkle tree of the log. We will reuse these restrictions to formulate inclusion proofs on that Merkle tree. Following [10], an inclusion

proof alone does not provide the recipient with any guarantees on the chain validity or existence.

Visibility of Chains. A monitor that inspects a log is usually interested in more than just a proof of inclusion of a pre-certificate. Think of the case where a monitor finds a potentially rogue pre-certificate entry included in the Merkle tree structure. Without a chain of certificates it is impossible to prove which CA issued the certificate and thus who is to blame. For this case, we adapt the visibility predicates to capture the pre-certificate and its chain together.

```
ChainVisible(L, fields, ca_cert, id_log, #time,
             sth_time) <=>
 (∃ chain different_time #t.
  LoggerComputesAddLog(L, id_log, fields, ca_cert,
                       chain, different_time)@t
  /* there has been an addition to id_log... */
  ∧ #t < #time /* ...prior to #time and... */
  ∧ ¬(sth_time = different_time)
  /* ...that addition was visible */
 )

NoChainVisible(L, fields, id_log, #time, sth_time) <=>
(¬(∃ different_time #t.
    LoggerComputesAddLogFields(L, id_log, fields,
                               different_time)@t
    ∧ ¬(sth_time = different_time)
    // there has never been a visible addition to the log
    ∧ #t < #time
    ))
 // or: for all additions, the chain is invalid
 ∨ ((∀ ca_cert different_time chain #t2 #t.
     LoggerComputesAddLog(L, id_log, fields, ca_cert,
                          chain, different_time)@t
     ∧ ¬(sth_time = different_time)
     ∧ #t < #time
     ∧ ChainCheck(chain)@t2
       ⇒ ChainInvalid(chain)@t2
    )
    ∧ (∀ different_time ca_cert chain #t.
    LoggerComputesAddLog(L, id_log, fields, ca_cert,
                         chain, different_time)@t
    ∧ #t < #time
    ⇒ (∃ #t2. ChainCheck(chain)@t2)
      )
   )
```

These last two predicates might appear odd at first sight. When checking a certificate,we will make use of them to distinguish whether a monitor can find a valid chain for a given pre-certificate entry in the log or not. By construction and if a monitor additionally verifies a found chain itself, a monitor is guaranteed to find a rogue certificate with a valid chain, if it exists in the log. This makes our model as expressive as a real-world CT monitor. For `NoChainVisible`, we add an additional disjunction that captures the case where all chains for a given pre-certificate are invalid. Note that while invalid entries could be used to explore potential misbehavior too, we are interested in the accountability perspective. If the chain is invalid, we are not able to precisely blame the responsible CA. Invalid chains on the log are considered a log violation. This vector has similar effects as equivocation: log misbehavior leads to cover up of CA misbehavior.

D.6 Fetching proofs and snapshots

We keep inclusion and append-only proofs fully abstract as trace property. This means we do not compute proofs that are then verified when other parties receive them, but use fresh variables as symbolic proofs. In order to be able to make use of proofs in the model, we use different trace properties that capture a snapshot of a log at the time of the proof. We say that the proof succeeds in the model if an actual proof would have worked for the situation of the log at the time of creating the proof. We axiomatically assert this relation with restrictions on the trace.

For an inclusion proof of entry x, this means that the proof succeeds in the model if, at the time of handing out the proof for some log, x is visible. Append-only proofs verify that between different snapshots of a log, no entries disappear. In Section D.7 and Section D.8, we present the details of necessary restrictions to model this approach specifically for inclusion and append-only proofs. This approach assumes, as is common for the symbolic model, perfect cryptography of all Merkle tree proofs.

Fetching new snapshots consists of three steps. First, the party that fetches the snapshot starts by creating a `ProofRequest` fact. Loggers can receive this fact and then hand out the current snapshot for their log, if they are honest, or, if they are corrupted, for one of the logs that they maintain, including forged ones.

```
rule ProofFetching_M:
 [ Fr(~session), !Monitor($M), !Logger($L, pkL) ]
 -[
  FetchLogSnapshot($M, $L, ~session),
  FetchLogSnapshotTime($M, $L, ~session, $now),
  Time($now)
 ]->
 [ TLS_M_to_L($M, $L, ~session, $now)
   /* Note: The established connection is not persistent
      such that new requests will be assigned a different
      session identifier */
 ]
```

We add a `~session` value to every proof request in order to assign every start of a proof to the proof itself. In doing so, we are able to formulate time restrictions on a proof run, e.g., by stating that a client initiates the proof request after it has received a corresponding SCT.

In the second step, a logger hands out the snapshot for a id_{log} of their choice. This is denoted by the `LoggerHandsOutSTH` action where id_{log} is free. id_{log} is dynamically instantiated and is therefore potentially attacker-controlled. We restrict that the instantiated id_{log} corresponds to a log that the logger maintains, i.e., the id exists. As honest loggers only maintain a single log, an attacker can only interfere here if the logger is corrupted. The fact contains the session of the TLS connection allowing us to later reconstruct the temporal sequence of actions, involved parties, and a freshly generated `~sth` as well as the current time.

The current STH can be seen as a commitment of the log with id_{log}. All underlying `$log_entries` required for verifying the STH are delivered separately.

```
rule LoggerSnapshot_M:
 [ !Logger_s($L, skL, pk(skL)), Fr(~sth)
   /* abstract representation of the STH */,
   TLS_M_to_L($M, $L, session, $time) ]
 -[
  LoggerPresentsView($L, $M, id_log, $now, ~sth, session),
```

```
  LoggerHandsOutSTH($L, ~sth, session),
  Time($now)
]->
[ !TLS_L_to_M($L, $M, $snapshot, ~sth, session, $time) ]
```

This rule is reused for all proofs involving the Merkle tree and monitors fetching all entries. This is due to our abstraction of proofs that we assume to be perfectly secure, which is typical for analyses in the DY model. We implicitly assume that loggers comply when presenting an STH and every underlying entry, as we cover malicious behavior separately. This assumption is relaxed when we turn to gossiping, where only the STH is shared, but not all underlying entries (Section 7).

Incoming proof requests and outgoing proofs are not implemented on the network to exclude the Dolev-Yao adversary. Assume the contrary and an adversary that can intercept old proofs and replay them later, while the logger behaves honestly. In these cases, an old snapshot might be used at a later point and disprove inclusion. A natural step would be to blame the logger in this case, as the presented proof is not valid, but the logger was honest the whole time. This is problematic when we aim at achieving accountability.

To highlight this limitation, we explicitly use a secure channel for these steps. Following the CT standard, the communication with a logger is implemented using HTTPS GET and POST requests, thereby excluding such attacks too. This secure channel has a session identifier that is used to relate the steps of the proof run.

In the third step, the handed-out snapshot is received and checked for the property of interest. We use different rules and restrictions representing every possible outcome of that check. We outline these in the next sections.

D.7 Inclusion Proof

We formulate inclusion proofs using the visibility predicates presented in Section D.5. The timepoint at which the log hands out the snapshot used in the proof, is then applied to the visibility predicate.

```
restriction inclusionProofFails:
 "∀ IdFetcher L fields ca_cert sth session #t1.
 InclusionProofFails(IdFetcher, L, fields, ca_cert, sth,
                     session)@t1
 ⇒ (∃ id_log time #t0.
      LoggerPresentsView(L, IdFetcher, id_log, time, sth,
                         session)@t0
      // Logger handed out an STH to the recipient...
      ∧ #t0 < #t1 // ...prior to the check
      ∧ EntryMissing(L, fields, id_log, #t0, time))"
      // ...and the entry is not visible in that view
```

In the successful variant the entry is instead visible.

```
restriction inclusionProofWorks:
"∀ [...] .
 InclusionProofSucceeds(id_fetcher,L,fields,ca_cert,
                        chain, sth, session)@t5
 ⇒ ∃ id id_log time #t2.
  LoggerPresentsView(L,id,id_log,time,sth,session)@t2
  ∧ #t2 < #t5
  ∧ EntryVisible(L,fields,ca_cert,chain,id_log,#t2,time)"
```

Depending on the action fact in the trace, we learn whether the proof can succeed.

D.8 Append-Only Proof

When we talk about the append-only property, we always refer to a pair of snapshots of a log. The proof shows that all entries in the older snapshot are also included in the more recent variant. This proof should not be possible if entries have been removed from the log in between.

We capture this behavior by introducing a rule that simply collects and stores a snapshot of the log and a second kind of rule that either proves or disproves the property based on all held snapshots. Similar to the inclusion proof, we use restrictions for that.

```
rule MonitorStoresSnapshot:
[ !TLS_L_to_M($L, $M, $snapshot, sth, session, $time),
  !Logger($L, pkL), !Monitor($M) ]
-[
 SnapshotStored($M, $L, session, sth),
 Time($now)
]->
[  ]
```

The implementation for a client doing the same is analogous. The formalization of an unsuccessful append-only proof is given below. Note that for a violation it is sufficient to find one pair that witnesses the violation.

```
restriction AppendOnlyFails:
"∀ sth session id L #t6.
AppendOnlyViolation(id, L, sth, session)@t6
/* Append-only violation found by id with violating L is
   only possible if a counterexample is found */
⇒
 (∃ fields ca_cert chain time t id_log1 id_log2 time2 sth2
     session2 #t0 #t1 #t2 #t3 #t4 #t5.
  LoggerComputesAddLog(L, id_log1, fields, ca_cert,
                       chain, t)@t0
  ∧ EntryVisible(L, fields, ca_cert, chain, id_log1,
                 #t2, time)
  /* there is an entry in the log that is visible at
     #t2 and time period time in id_log1 */
  ∧ FetchLogSnapshot(id, L, session2)@t1
  /* someone starts fetching a log view */
  ∧ LoggerPresentsView(L, id, id_log1, time, sth2,
                       session2)@t2
  /* The Logger presents the view for id_log1 */
  ∧ SnapshotStored(id, L, session2, sth2)@t3
  /* this view is received and stored by id */
  ∧ FetchLogSnapshot(id, L, session)@t4
  /* a second view is fetched by id (later) */
  ∧ LoggerPresentsView(L, id, id_log2, time2, sth,
                       session)@t5
  /* the logger possibly presents a different log */
  ∧ EntryMissing(L, fields, id_log2, #t5, time2)
  /* one entry that was visible in the past is now not
    visible anymore -> clear append-only violation */
  ∧ #t0 < #t1 ∧ #t1 < #t2 ∧ #t2 < #t3 ∧ #t2 < #t4
  ∧ #t4 < #t5 ∧ #t3 < #t5 ∧ #t5 < #t6)"
```

For verifying the append-only property given an up-to-date snapshot of the log, we take all existing snapshots a party has retrieved and stored in the past into account. The formalization is analogous to the previous one by asserting that every entry that was visible before should still be visible.

E CT with Receipts (Details)

So far, we have excluded monitor misbehavior from our accountability analysis. CT does not provide us with a mechanism suited

for accountability tests, as a party distinct from the monitor cannot verify whether the monitor purposefully ignored a rogue certificate, let alone whether a monitor even saw the certificate in the first place.

To address this limitation, we further extend our model of SCT Auditing with *receipts* issued by the monitor back to the logger. The receipt proves to a logger (and any other party in possession of it) that the monitor has seen a specific snapshot of the log. SCT Auditing is in particular of interest here, as we were able to show that accountable authenticity can be achieved in SCT Auditing given honest monitors, while allowing corrupted loggers. With this receipt approach we aim to achieve accountable authenticity even in the presence of corrupted monitors.

We adapt our model by giving a key pair to every monitor and model the signing process.

```
rule signReceipt:
let
 receipt = sign(<$M, chain>, skM)
in
[ CraftReceipt($M, chain),
  !MonitorKey_s($M, pkSKM, skM)
]
−[ Time($now) ]→
[ !TLS_M_to_C(receipt)
  /* We already assumed a TLS connection
     between client and monitor before */ ]
```

`CraftReceipt` is a fact emitted by the SCT Auditing rules when a monitor receives an audited certificate chain from a client. The receipt is a simple signature over that received chain and the monitor's identifier. The signature is then sent back to the client via a TLS channel, modeled by the `TLS_M_to_C` fact.

The client then, if necessary, is able to forward the receipt to the domain owner.

```
rule domainOwnerBlamesMonitor:
let
 ca_fields =
    <$rootCA, $CA, pkCA, 'pubKey-CA', snCA, $i0, $j0>
 pub_fields =
    <$CA, $id, pkID, 'pubKey', sn, $i, $j, sct1, sct2>
 fields = <$CA, $id, pkID, 'pubKey', sn, $i, $j>
 chain = <<pub_fields, pub_sig>, <ca_fields, ca_sig>>
in
[ In(<$M, receipt, chain>), !MonitorKey($M, pkSKM),
  !TrackSubject_Contract($M, $idServer),
  !Root_CA($rootCA, pkRootCA) ]
−[
  Eq(verify(receipt, <$M, chain>, pkSKM), true()),
  Eq(verify(pub_sig, pub_fields, pkCA), true()),
  Eq(verify(ca_sig, ca_fields, pkRootCA), true()),
  DO_MonitorBlame($M, $CA, ca_fields, fields),
  Asserts($id,'This_monitor_did_not_react', $M,<fields>)
 ↳ ,
  Asserts($id, 'That_intermediary_CA_signed_the_
 ↳ following_incorrect_statement_and_the_validation_
 ↳ chain_says_the_following_about_that_CA.', $CA, <
 ↳ fields, ca_fields>),
  // external validator style assertions for authenticity
  RogueCert($CA, fields),
  Time($now)
]→
[ ReceiptChecked($M, fields, ca_fields) ]
```

In general, we assumed in SCT Auditing that every received certificate is checked. To allow corrupted monitors to diverge from this behavior, we allow them to *sleep* on an entry, i.e., seeing the entry but not acting on it. In this case, the `DO_MonitorBlame` action can be reached.

Further rules, omitted here, consider the variants where we cannot blame a monitor. This is the case if either the monitor acted benignly when confronted with the rogue certificate, i.e., raised an alert to the domain owner, or if the monitor did not need to react, as the certificate was benign.

E.1 Accountability Analysis

In this scenario, we assume that a domain owner finds a rogue certificate that was actively used. As the domain owner has access to the ground truth, accountable authenticity via an external check is easily achieved, as seen in Section 4. But, what if the domain owner contracted a monitor service to do the same thing, but it raised no alarm?

The CT-receipt model allows tests that detect monitor misbehavior and, combined with previous test components concerning the intermediate or root CA misbehavior, provides us with accountable authenticity against untrusted monitors. This allows, e.g., for probabilistic checking of the monitor, or to use multiple monitors to validate each other (which is essentially the same test).

```
test receipt_1:
 "∃ M id fields rootCA ca_fields #t #t1 #t2 #t3.
 SCTReceiptCheck(M, fields)@t
 ∧ Asserts(id, 'That_intermediary_CA_signed_the_
 ↳ following_incorrect_statement_and_the_validation_
 ↳ chain_says_the_following_about_that_CA.', CA, <
 ↳ fields, ca_fields>)@t1
   // CA blamed
 ∧ Asserts(id, 'This_monitor_did_react', M, <fields>)@t2
   // monitor not blamed
 ∧ Asserts(rootCA, 'I_signed_this_intermediate_CA.', CA,
          <ca_fields>)@t3"
   // CA is liable

test receipt_2:
 "∃ M id fields CA ca_fields #t #t1 #t2 #t3.
 SCTReceiptCheck(M, fields)@t
 ∧ Asserts(id, 'That_intermediary_CA_signed_the_
 ↳ following_incorrect_statement_and_the_validation_
 ↳ chain_says_the_following_about_that_CA.', CA, <
 ↳ fields, ca_fields>)@t1
   // CA blamed
 ∧ Asserts(id, 'This_monitor_did_react', M, <fields>)@t2
   // monitor not blamed
 ∧ Asserts('CCADB', 'No_root_CA_signed_this_CA.', CA,
           <ca_fields, rootCA>)@t3"
  // rootCA is liable
```

Two more analogous tests blame the intermediate CA instead, as seen before.

Together they construct the following lemma, which verifies.

Listing 2: SCT Auditing with receipts provides acc. auth.

```
lemma A_Receipt:
receipt_1, receipt_2, receipt_3, receipt_4
accounts for
 "∀ CA fields rootCA ca_fields L1 L2 M #t0 #t1.
 SCTReceiptCheck(M, fields)@t0
 ∧ BrowserAcceptsChain(CA, fields, rootCA, ca_fields,
```

```
                    L1, L2)@t1
  ⇒ (∃ CA #t. GroundTruth(CA, fields)@t)"
```

E.2 Discussion

Using receipts issued by monitors to loggers, we were able to design tests that allow us to blame corrupted monitors that purposefully ignored rogue certificates. Together with previous tests blaming corrupted CAs, we achieve accountable authenticity even in the presence of corrupted monitors. Receipts illustrate a possible mechanism for holding monitors accountable for their actions. We moved multiple tests to the domain owner here, which is a natural choice as the domain owner has access to the ground truth and is particularly interested in correct behavior of the contracted monitor.

We relied on the fact that the domain owner can access such a receipt, e.g., because the client forwards it. It is not straightforward how to ensure that the domain owner actually receives such receipts in practice. While domain owners run their own web server, i.e., they are addressable, sending the receipt to the *correct* web server boils down to the authenticity problem we started with. A more realistic perspective is that domain owners purposefully act as clients, to probabilistically test for misbehavior. This would still need 'normal' clients to require receipts, but they would not need to always forward them.

Additionally, we assumed that the monitor issues such receipts for every received certificate. While a corrupted monitor could choose not to issue a receipt, note that a client could always demand a receipt or otherwise refuse to accept the certificate to protect themselves.

As complexity on the domain-owner side increases, one could raise the argument that a monitor distinct from the domain owner is not necessary anymore. Recall that everyone can become a monitor and self-run monitors have the advantage that the running party immediately trusts this monitor. However, SCT Auditing assumes the presence of a powerful monitor that has access to all certificates that clients eventually see. We already discussed in Section 9 that self-run monitors with these capabilities are not realistic. A mechanism to hold monitors accountable is in particular interesting if domain owners have to rely on such a powerful third-party monitor.

F Modeling STH Gossiping (Details)

In this section, we will introduce the additions to our model and reformulate properties to apply them to STH gossiping.

If a monitor wants to ensure that it can see the entries behind a gossiped STH, it additionally needs to run an append-only proof between the two snapshots it considers.

We use similar restrictions to axiomatically assert append-only proofs (Section D.8), but this time the gossiped proof is not necessarily handed out to the monitor but rather to any party to reflect the nature of gossiping.

```
rule monitorFindsAppendOnlyViolation_Gossiped:
[ !LoggerDict($M, $L),
  /* Logger is monitored */
  !TLS_C_to_M($C, $M, $gos_snapshot, gos_sth, gos_sess),
  /* Gossiped snapshot submitted by a client */
  !TLS_L_to_M($L, $M, $snapshot, sth, sess, $time)
  /* Own snapshot of the logger that the monitor holds.
     We assume that this snapshot is freshly fetched
     (see  restriction below), s.t. the monitor can
     verify that it can see as much as the client. */
  ]
-[
 Gos_AppendOnlyViol($M,$L,sth,sess,gos_sth,gos_sess),
 /* see restriction */
 Asserts($M, 'The log has violated the append-only
 ↳ property between these two snapshots.', $L,
         <gos_sess, sess>),
 MonitorReceivesGossip($M, $L, gos_sess, sess),
 Time($now)
]->
[ AppendOnlyChecked($M, gos_sess, sess) ]
```

The corresponding restriction relaxes the assumption that both snapshots have been handed out to the monitor to reflect the idea of gossiping.

```
restriction appendOnlyFails_Gossiped:
"∀ M L session sth gossiped_sth gossiped_session #t5.
 Gos_AppendOnlyViol(M, L, sth, session, gossiped_sth,
                    gossiped_session)@t5
 ⇒
  (∃ id1 fields ca_cert chain time t time2 id_log id_log_2
   #t0 #t1 #t2 #t3 #t4.
   LoggerComputesAddLog(L, id_log, fields, ca_cert,
                        chain, t)@t0
   ∧ EntryVisible(L,fields,ca_cert,chain,id_log,#t2,time)
   // There is an entry that was visible...
   ∧ FetchLogSnapshot(id1, L, gossiped_session)@t1
   ∧ LoggerPresentsView(L, id1, id_log, time, gossiped_sth,
                        gossiped_session)@t2
   // ... for a given proof at #t2
   ∧ FetchLogSnapshot(M, L, session)@t3
   ∧ LoggerPresentsView(L,M,id_log_2,time2,sth,session)@t4
   /* second view is handed out to the monitor directly
      (only one of both views is gossiped as assumed) */
   ∧ EntryMissing(L, fields, id_log_2, #t4, time2)
   /* one entry that was visible in the gossiped view is
      now not visible anymore -> clearly a violation */
   ∧ #t0 < #t1 ∧ #t1 < #t2 ∧ #t2 < #t3
   ∧ #t3 < #t4 ∧ #t4 < #t5
)"
```

The other rules consider the opposite case and are analogously adapted.